

张量手册

Thomas Dybdahl Ahle

2026 年 6 月 16 日

第一章 引言

这是什么？ 这些页面是一本张量导览，使用“张量图”的视觉语言。为了展示这种方法的普适性，我尽量紧跟传奇的《Matrix Cookbook》。因此，正文大多是关于张量及其相关内容的一组事实（恒等式、近似、不等式、关系，等等）。你不会在这里看到太多原始手册之外的结果，但希望这些图示能为你提供一种新的理解和欣赏方式。

这是一个持续进行的项目： 《Matrix Cookbook》是一本很长的书，并不是每一部分都同样适合用图示表达。因此我选择跳过某些部分，并缩短另一些部分。也许将来，我或其他人会进一步扩展覆盖范围。

例如，我们覆盖了关于线性组合的期望和高斯矩的全部结果，但跳过了一般多元分布那一节。我也不得不稍微重排材料，以避免一开始就把所有记号都介绍完。

复数矩阵与协方差 张量图（或张量网络）目前最常见于量子物理，但这不是一本给物理学家的书。《Matrix Cookbook》是一本给工程师的书，尤其是机器学习领域，而在这里复数并不常见。没有复数，我们就不必担心复共轭，这简化了转置，也去掉了协变与逆变张量的需要。如果你是物理学家，你大概更需要一本关于张量分析的书。

Tensorgrad 张量图的符号化特性使它们非常适合符号计算。

张量图记号的优点： 与其他记号相比，张量图记号有很多好处：

诸如迹、张量积或张量缩并之类的操作，都可以不借助额外记号而直接表达。指标和张量的名称往往可以省略。这既省时又简化记号，尤其适合那些主要只是用于求和的内部指标。复杂缩并网络所得到张量的阶数可以一眼看出：它就是未配对连线的数量。比如，一个所有连线都接上的张量网络，无论多复杂，结果都必然是一个标量。

词源 “tensor”一词源自拉丁语 *tensio*，意为“张力”或“伸展”，又来自动词 *tendere*，意为“拉伸”或“延展”。它最早在 19 世纪中叶由 William Rowan Hamilton 在四元数

研究中引入数学语境，当时它指的是四元数的模长。后来 Gregorio Ricci-Curbastro 和 Tullio Levi-Civita 在发展张量微积分时确立了“tensor”的现代用法；这套框架把标量、向量和矩阵的概念推广到了更复杂的多维对象。[1, 10].

目录

第一章 引言	1
1.1 张量图	8
1.2 复制张量	9
1.3 张量求和	10
1.4 转置	11
1.4.1 高阶情形	12
1.5 迹	12
1.5.1 高阶迹	12
1.6 特征值	13
1.7 对称与对称化	13
1.8 协变与逆变	14
1.9 练习	14
第二章 简单导数	15
2.1 矩阵、向量与标量形式的导数	15
2.1.1 一阶	15
2.1.2 二阶	16
2.1.3 高阶	17
2.2 迹的导数	17
2.2.1 一阶	17
2.2.2 二阶	18
2.2.3 高阶	19
2.3 练习	20
第三章 Kronecker 积与 Vec 算子	21
3.1 展平	21

目录	4
3.2 Kronecker 积	22
3.3 Vec 算子	23
3.4 Kronecker 向量积	24
3.5 一般矩阵化	24
3.5.1 Lyapunov 方程	25
3.5.2 封装求和	25
3.6 Hadamard 积	25
3.7 Khatri-Rao 积	26
3.7.1 拼接	27
3.8 导数	28
3.9 练习	28
第四章 函数	30
4.1 链式法则	32
4.1.1 Hessian 链式法则	32
4.1.2 带广播的链式法则	33
4.1.3 多输入函数	34
4.2 例子	34
4.2.1 已知导数	34
4.2.2 伪线性形式	34
4.2.3 迹恒等式	35
4.2.4 Taylor	36
4.3 练习	36
第五章 统计与概率	40
5.1 矩的定义	40
5.1.1 线性组合的期望	40
5.1.2 线性形式	41
5.1.3 二次型	41
5.1.4 三次型	42
5.2 累积量	43
5.2.1 四次型	44
5.3 加权标量变量	45
5.4 高斯矩	45
5.4.1 高斯分部积分	45

目录	5
5.4.2 三次型	46
5.4.3 四次型的均值	46
5.4.4 高斯混合	47
5.4.5 导数	48
5.5 向量与张量的自由概率	48
5.5.1 向量的自由累积量	48
5.5.2 矩阵的自由累积量	49
5.5.3 张量自由累积量	50
5.5.4 张量 Wick 定理	51
5.5.5 张量自由性下的可加性	52
5.5.6 张量 Wishart 矩阵的谱	53
5.5.7 张量网络与自由概率	53
5.5.8 高阶张量示例	54
5.5.9 D 元 Catalan 数	55
5.5.10 Melon 型图	56
5.6 练习	57
第六章 行列式与逆矩阵	59
6.1 行列式	59
6.2 逆矩阵	60
第七章 高级导数	61
7.1 向量范数的导数	61
7.1.1 二范数	61
7.2 矩阵范数的导数	62
7.3 结构化矩阵的导数	62
7.3.1 对称矩阵	62
7.3.2 对角矩阵	62
7.3.3 Toeplitz 矩阵	62
7.4 行列式的导数	62
7.5 一般形式	62
7.6 线性形式	62
7.7 平方形式	62
7.8 来自 Stack Exchange	62
7.9 逆矩阵的导数	62

目录	6
7.9.1 迹恒等式	62
7.10 特征值的导数	63
7.11 练习	63
第八章 特殊张量	64
8.1 快速 Kronecker 乘法	64
8.2 Hadamard 变换	64
8.3 Fourier 变换	65
8.3.1 Vandermonde 矩阵	67
8.3.2 块矩阵	68
8.3.3 对称、斜对称/反对称	69
8.3.4 Toeplitz 矩阵	69
8.3.5 单位矩阵、置换与移位	69
第九章 分解	70
9.1 高阶奇异值分解	70
9.2 秩分解	70
9.2.1 边界秩	70
9.3 快速矩阵乘法	71
第十章 机器学习应用	73
10.1 最小二乘	73
10.2 交叉熵损失的 Hessian 矩阵	73
10.3 卷积神经网络	73
10.4 Transformer / 注意力	73
10.5 张量草图	73
10.6 强化学习	73
第十一章 张量算法	74
11.1 张量缩并顺序	74
11.1.1 算法	75
第十二章 Tensorgrad	76
12.1 同构	76
12.1.1 在乘积中	76
12.1.2 在求和中	77

目录	7
12.1.3 在求值中	78
12.1.4 在变量中	78
12.1.5 在常量中	79
12.1.6 在函数中	79
12.1.7 在导数中	79
12.1.8 其他	79
12.2 重命名	79
12.3 求值	79
12.3.1 乘积	81
12.4 化简规则	81
第十三章 附录	85

1.1 张量图

张量图是简单的图（或“网络”），其中节点表示变量（例如向量或矩阵），边表示缩并（例如矩阵乘法或内积）。下表展示了一些基本操作如何用张量图写出：

点积	$a-b$	$y = \sum_i a_i b_i$	$[\cdots] \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}$	$= y$
外积	$-a \quad b-$	$Y_{i,j} = a_i b_j$	$\begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} [\cdots]$	$= -Y-$
矩阵-向量	$-A-b$	$y_i = \sum_j A_{i,j} b_j$	$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}$	$= -y$
矩阵-矩阵	$-A-B-$	$Y_{i,k} = \sum_j A_{i,j} B_{j,k}$	$\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$	$= -Y-$

我们把向量和矩阵看作 1 阶和 2 阶张量。阶数对应于上面 $[\cdots]$ 可视化中的维数，例如，向量是一维的数字列表，而矩阵是二维的数字网格。阶数也决定了张量图中表示该变量的节点的度数。

当我们处理 3 阶及更高阶张量时，图记号会变得更有趣。一个 3 阶张量可以看作一个数字立方体，或者一叠矩阵。例如，我们可以把它写成 $T \in \mathbb{R}^{n \times m \times k}$ ，于是对 $i = 1 \dots n$ ， $T_i \in \mathbb{R}^{m \times k}$ 是一个矩阵。当然，我们也可以沿其他轴切分 T ，因此 $T_{:,j} \in \mathbb{R}^{n \times k}$ 和 $T_{:, :, \ell} \in \mathbb{R}^{n \times m}$ 也都是矩阵。

矩阵有两条伸出的边，意味着可以用两种方式让向量与它相乘：从左边乘，即 $x^T M$ ；或者从右边乘，即 Mx 。在图记号中，我们只写作 $x-M-$ 和 $-M-x$ 。3 阶张量有三条边，因此可以用三种方式让它与一个向量相乘：

$$\begin{array}{c} | \\ \diagup T \diagdown \\ x \end{array} \quad \text{和} \quad \begin{array}{c} | \\ x \diagup T \diagdown \end{array} \quad \text{和} \quad \begin{array}{c} x \\ | \\ \diagup T \diagdown \end{array}$$

要完全精确地说明每一种写法的含义，我们应该给边加上标签。例如，我们会写成 $\begin{array}{c} | \\ \diagup T \diagdown \\ x \end{array}$ 来表示矩阵 $\sum_i T_i x_i$ 。不过，相关的边通常能从上下文中看出；这正是张量图记号比 Einstein 求和记号更清爽的原因之一。

$$Y_{i,j} = \sum_{k,l,m,n,o} A_{i,k} B_{l,n,o} C_{j,k,l,m} D_{m,n} E_o \Leftrightarrow \begin{array}{c} \diagup^i \\ j \diagdown Y \end{array} = \begin{array}{c} \diagup^i A \\ k \diagdown \\ j \diagup C \end{array} \begin{array}{c} \diagdown^l B \\ | \\ m \diagup D \end{array} \begin{array}{c} \diagdown^o E \\ | \\ n \diagup \end{array}$$

张量图的核心原则是：边缩并满足结合律。这意味着你可以按任意喜欢的顺序缩并任意边。从上面的求和表示也能看出这一点，因为其中对 k, l, m, n 的求和可以按任意顺序重排。

不同缩并顺序的计算代价可能相差很大。遗憾的是，寻找最优顺序在计算上并不容易。关于寻找最佳缩并顺序的算法以及近似缩并方法，见第 11.1 节。

注意，张量图并不总是连通的。我们已经看到，两个向量的外积可以写成 $-a \quad b-$ 。从求和表示来看这很自然：没有边就意味着没有求和。因此这里 $y_{i,j} = a_i b_j$ ，这正是外积 $y = a \otimes b$ 。

1.2 复制张量

一个特别重要的张量是“复制”张量，也称为“对角”张量、“Kronecker delta”张量或“spider”张量。最简单的版本是全 1 向量，我们把它写作 $\circ-$ 。也就是说 $\circ_i = 1$ 。一般的 n 阶版本在对角线上取值为 1，在其他位置取值为 0：

$$\circ_{i,j,k,\dots} = \begin{cases} 1 & \text{if } i = j = k = \dots \\ 0 & \text{otherwise} \end{cases}$$

或者，使用 Iverson 记号，¹ $\circ_{i,j,k,\dots} = [i = j = k = \dots]$ 。可以看到，2 阶复制张量 $-\circ- = I$ 只是单位矩阵，因此可以像这样直接从图中删去：

$$-A-\circ-B- = -A-B-$$

高阶复制张量非常有用，因为它们让我们能够把简单的张量图变成超图。一个简单的用法是表示对角矩阵 D_a ，它在对角线上是 a ，其他位置是 0。我们可以写成

$$D_a = \begin{array}{c} | \\ \circ \\ \swarrow \searrow \\ a \end{array}$$

为什么？因为 $(D_a)_{i,j} = \sum_k \circ_{i,j,k} a_k = \sum_k [i = j = k] a_k = [i = j] a_i$ 。类似地，Hadamard 积 $(a \circ b)_i = a_i b_i$ 可以写成

$$a \circ b = \begin{array}{c} | \\ \circ \\ \swarrow \searrow \\ a \quad b \end{array}$$

现在，让我们看看为什么大家都喜欢复制张量：用“复制张量操作”来证明恒等式 $D_a D_b = D_{a \circ b}$ ：

$$D_a D_b = \begin{array}{c} | \quad | \\ \circ - \circ \\ \swarrow \searrow \quad \swarrow \searrow \\ a \quad b \end{array} = \begin{array}{c} \swarrow \quad \swarrow \\ \circ \\ \searrow \quad \searrow \\ a \quad b \end{array} = \begin{array}{c} \swarrow \quad \swarrow \\ \circ \\ \searrow \quad \searrow \\ a \quad b \end{array} = D_{a \circ b}.$$

¹ 对一个逻辑命题 P ，我们定义 $[P] = \begin{cases} 1 & \text{if } P \\ 0 & \text{otherwise} \end{cases}$ 。

你可以用求和表示来验证这一点。

这里起作用的一般规则是：任何由复制张量组成的连通子图都可以合并成一个复制张量。有时我们甚至足够幸运，化简后还会留下一个可以删去的单位矩阵：

$$S \text{---} \text{---} \text{---} T = S \text{---} T = S \text{---} T$$

唯一需要稍微小心的是结果张量为 0 阶的时候。根据你如何定义 0 阶复制张量 $\circ$ ，恒等式 $\circ \text{---} \circ = \circ$ 可能成立，也可能不成立。

在普通向量记号中许多需要特殊记号的构造（例如对角矩阵或 Hadamard 积），都可以用复制张量统一起来。在《Matrix Cookbook》中，他们定义了 4 阶张量 J ，它满足 $J_{i,j,k,l} = [i = k][j = l]$ ，我们会把它写成 $J = \text{---}\text{---}\text{---}\text{---}$ ，并且满足例如 $\frac{dX}{dX} = J$ 。使用“张量积”，你可以写成 $J = I \otimes I$ 。注意， J 不同于 4 阶复制张量 $\bowtie$ 。

1.3 张量求和

张量积可以表示任何线性函数。也就是说，满足 $f(ax, by) = abf(x, y)$ 的函数都可以这样表达。不幸的是，并非所有张量上的操作都是线性的。即使像两个向量之和 $x + y$ 这样简单的运算，也无法用一个简单的缩并图来表示。（注意这不是线性的，因为 $ax + by \neq ab(x + y)$ 。）

为了处理这个重要操作，Penrose 建议直接把两个图写在一起，中间加一个加号，例如 $-x + -y$ 。注意这本身仍然是 1 阶张量，尽管看起来像是有两条自由边。如果我们想把这个和与另一个张量相乘，可以用括号写成 $-M - (-x + -y)$ 。

在处理求和时使用命名边会很有帮助，这样更容易看清哪些边彼此对应。求和与张量积之间有很好的相互作用，满足一种广义的分配律：

$$\left(\begin{array}{c} \text{---}^i \\ \text{---}^k \\ \text{---}^j \end{array} \right) \left(\begin{array}{c} \text{---}^i \\ \text{---}^k \\ \text{---}^j \end{array} \right) M + \left(\begin{array}{c} \text{---}^k \\ \text{---}^j \end{array} \right) V = \begin{array}{c} \text{---}^i \\ \text{---}^k \\ \text{---}^j \end{array} M + \begin{array}{c} \text{---}^k \\ \text{---}^j \end{array} V$$

当我们相加的张量边数不同，或者边名不同的时候，可以使用“广播 (broadcasting)”。假设我们想把一个矩阵 M 和一个向量 x 相加。这到底是什么意思？如果我们想把 x 加到 M 的每一行上，可以写成 $\frac{i}{j} M + \frac{i}{j} x$ 。这是因为 $\frac{i}{j} x$ 这是 x 与全 1 向量的外积，得到的是一个每一行都相同的矩阵。类似地，如果我们想把 x 加到每一列上，可以使用这个矩阵 $\frac{i}{j} x$ 。

注意，处理张量时我们通常不谈“行”或“列”，而只是使用张量的边名（有时也叫轴）。当使用命名边时，像“转置”这样的经典向量记号操作也可以去掉。矩阵 X^T 只是把左边和右边的边交换一下而已。但如果边已经命名，我们就根本不必再去追踪“边在什么位置”了。

1.4 转置

在经典矩阵记号中，转置会交换矩阵的指标，因此

$$(A^T)_{ij} = A_{ji}.$$

在张量图记号中，我们有两种选择，取决于是否要让边的位置具有意义。当边位置有意义时，我们通常让“左边”对应第一个指标，让“右边”对应第二个指标。因此转置就需要把边翻转：

$$(-A-)^T = \overline{\underbrace{A}} = -A^T-.$$

有些作者喜欢用把张量上下翻转的记号 $-V-$ ，作为交换左右边的更简洁写法。

在实际计算中，追踪边的位置很容易出错。更稳妥的做法是给张量的“输出”命名，并让转置去重命名这些边：

$$({}^i-A-j)^T = {}^{i-j}A-^i$$

重命名可以通过乘上单位矩阵来完成：

$$({}^i-A-j)({}_j-o-k) = {}^i-A^{j-k},$$

但必须小心，因为重叠的边名会让乘法失去结合性。例如：

$$[({}^i-A-j)({}_j-o-k)]({}_k-o-j) = {}^i-A-j \neq ({}^i-A-j)[({}_j-o-k)({}_k-o-j)] = {}^i-A-j \circledcirc,$$

其中 $\circledcirc$ 等于矩阵维数。在 Tensorgrad 中，我们通过要求任何一个边名在任意乘积中最多出现两次来解决这个问题。

为了转写《Matrix Cookbook》，使用“位置有意义”的记号更方便。我们注意到如下恒等式：

$$\begin{aligned} (A^T)^T &= A & \overline{\overline{A}} &= -A- \\ (A+B)^T &= A^T + B^T & \overline{(-A- + -B-)} &= \overline{-A-} + \overline{-B-} \end{aligned} \quad (4)$$

$$(AB)^T = B^T A^T \quad \overline{\overline{A-} \overline{B-}} = \overline{\overline{V-}}_{-B-} = -B-V- \quad (5)$$

1.4.1 高阶情形

可以把转置的思想推广到高阶张量。这要把边划分为“输入”和“输出”边，更常见地说，就是“逆变”边和“协变”边。关于这一点的更多内容，见本章末尾的小节。

如果 $A^T = A$ ，我们就说矩阵是对称的。对于高阶张量，对称的方式有很多种。更多内容见第 1.7 节。

1.5 迹

方阵的“迹”定义为 $\text{Tr}(A) = \sum_i A_{ii}$ 。在张量图记号中，这对应一条自环边：



。《Matrix Cookbook》中有一系列使用迹的恒等式。我们用张量图把它们重写出来：

$$\sum_{i=1}^n A_{ii} = \text{Tr}(A) = \text{Tr}(AI) \quad \text{Diagram: } \text{Circle with } A \text{ and } I \text{ (identity) connected} \quad (11)$$

$$\text{Tr}(A) = \text{Tr}(A^T) \quad \text{Diagram: } \text{Circle with } A \text{ and } A^T \text{ (transpose) connected} \quad (13)$$

$$\text{Tr}(AB) = \text{Tr}(BA) \quad \text{Diagram: } \text{Circle with } A \text{ and } B \text{ connected in two ways} \quad (14)$$

$$\text{Tr}(A+B) = \text{Tr}(A) + \text{Tr}(B) \quad \text{Diagram: } \text{Circle with } A \text{ and } B \text{ added} \quad (15)$$

$$\begin{aligned} \text{Tr}(ABC) &= \text{Tr}(BCA) \\ &= \text{Tr}(CAB) \end{aligned} \quad \text{Diagram: } \text{Circle with } A, B, C \text{ in different orders} \quad (16)$$

$$\begin{aligned} a^T a &= \text{Tr}(aa^T) \\ a - a &= \text{Tr}(-a \ a -) \\ &= \text{Diagram: } \text{Circle with } a \text{ and } a \text{ connected} \end{aligned} \quad (17)$$

1.5.1 高阶迹

对于高阶张量，我们仍然把迹定义为对角元素之和。也就是说，对于一个 n 阶张量 A ，迹就是它与 n 阶复制张量的乘积：

$$\text{Tr}(\underset{\nearrow}{T}) = \underset{\nearrow}{T} \text{Diagram: } \text{Circle with } T \text{ and its copy connected}.$$

在量子力学中，常会用到“部分迹”，我们可以用指标记号来定义：

$$\text{Tr}_{i,j}(\overset{|^k}{T}_{\underset{|}{i}\underset{|}{j}}) = \overset{|^k}{T}_{\bigcirc}.$$

当然，在张量图里，我们也可以不命名指标就使用部分迹。我们甚至不必去想所做的缩并到底是不是迹。

1.6 特征值

特征值和特征向量是线性代数中的基本概念，在张量网络理论中也有重要应用。张量图记号可以把这些概念直观地表示出来。

对于矩阵 A ，如果存在非零向量 v 和标量 λ 使得 $Av = \lambda v$ ，那么 λ 就称为 A 的一个特征值，而 v 是相应的特征向量。在张量图记号中，把它的特征分解写成 $A = Q\Lambda Q^{-1}$ 很方便，其中 Q 的列是 A 的特征向量， Λ 是由特征值组成的对角矩阵。因此：

$$-A- = -Q-\underset{\lambda}{\circ}-Q^{-1}- \quad (1.1)$$

矩阵的迹等于其特征值之和。我们可以用张量图表示这一关系：

$$\text{Tr}(A) = \text{A} = \text{Q}-\underset{\lambda}{\circ}-\text{Q}^{-1} = \text{Q}-\underset{\lambda}{\circ} = \underset{\lambda}{\circ} = \sum_i \lambda_i \quad (12)$$

1.7 对称与对称化

张量的一个常见性质是对称性。对矩阵来说，对称性意味着 $A^T = A$ 。对张量来说，则意味着交换其指标不会改变它的值。典型例子是协方差矩阵或 Hessian 矩阵，但某些高阶统计矩，比如三阶或四阶矩张量，也可能具有对称性。有时这种对称性只相对于某个特定群成立，但大多数时候是相对于边的所有置换成立。复制张量就是一个完全对称张量的简单例子。

有时我们会通过对所有指标置换求和来对张量进行对称化。我们用一条波浪线覆盖被对称化的边来表示：

$$\text{Z} = \text{=} + \text{X}, \quad \text{Z} = \text{=} + \text{X} + \text{X} + \text{X} + \text{X} + \text{X}$$

例如，如果 A 是一个方阵， $A + A^T = \text{Z}A = -A + \text{A}$ 。还有一个互补概念是反对称化（或斜对称化），此时我们对所有置换按适当符号求和。例如，一

个 2 阶斜对称矩阵 A 满足 $A_{ij} = -A_{ji}$ 。我们可以用一条扁平的粗线表示反对称化：
 $\text{---}\overbrace{A}^{\text{---}}\text{---} = -A - \text{---}\overbrace{A}^{\text{---}}\text{---}$ 。在更高阶的情形下，每一项的符号由置换的奇偶性决定。

这两种张量都是幂等的，因为对一个已经对称的张量再做对称化不会改变它：

$$\text{---}\overbrace{\text{---}\overbrace{A}^{\text{---}}\text{---}}^{\text{---}}\text{---} = \text{---}\overbrace{A}^{\text{---}}\text{---}, \quad \text{---}\overbrace{\text{---}\overbrace{\text{---}\overbrace{A}^{\text{---}}\text{---}}^{\text{---}}\text{---}}^{\text{---}}\text{---} = \text{---}\overbrace{A}^{\text{---}}\text{---}.$$

1.8 协变与逆变

在物理学中，经常需要在不同坐标系之间变换。这类变换往往会以不同方式作用在矩阵的左侧和右侧，也会类似地作用于行向量和列向量。对于张量，这就推广为协变与逆变的概念。借助这些概念，我们需要跟踪“输入”和“输出”向量。我们还需要区分诸如单位矩阵 $\text{---}\circ\text{---}$ 、 ∞ ，以及 $\propto$ 。

这个概念让我们可以通过把输入边连接到输出边，像处理矩阵和向量那样在一定程度上“代数地”组合张量。不过，对更复杂的张量来说，我们需要知道具体该连接哪些边，而这时指标记号更有用。

在计算机科学和机器学习中，协变与逆变的概念通常并不实用，所以本书不会使用它。

1.9 练习

练习 1. 给定一系列矩阵 $A_1, A_2, \dots, A_n \in \mathbb{R}^{n \times n}$ ，以及向量 $v_1, v_2, \dots, v_n \in \mathbb{R}^n$ ，请用张量图画出由向量 $A_1 v_1, A_2 v_2, \dots, A_n v_n$ 组成的矩阵。

练习 2. 用张量图表示两个矩阵的 Hadamard 积（逐元素乘法）。它与普通矩阵乘法有何不同？（关于这一点，见 3.6。）

练习 3. 用张量图表示矩阵 $A = U \Sigma V^T$ 的 SVD。这与特征分解的图示相比如何？如何把它推广到高阶张量？（在第 9.1 节我们会看到更多内容。）

第二章 简单导数

关于一个张量的导数，就是关于这个张量每个元素的导数组成的集合。我们可以构造一个形状为 $\text{shape}(T) \cup \text{shape}(U)$ 的张量来记录 $\frac{dT}{dU}$ 。例如，如果 T 是一个 3 阶张量，而 U 是一个 2 阶张量，我们把 dT/dU 画成

$$\frac{dT}{dU} = \text{Diagram: A circle with a dot on its right side. Two lines extend from the dot: one goes up and to the right, the other goes down and to the right. Inside the circle is the letter 'T'.$$

这个记号沿用 Penrose 的做法。圆圈上黑点引出的两条额外连线，使得该导数成为一个 5 阶张量。导数的阶数会以这种方式增长，正是张量一开始会自然出现的主要原因之一。

当边数不太多时，我们会使用下面这种简单的行内记号：

$$\succ(T)\prec$$

《Matrix Cookbook》把单项矩阵 $J^{i,j} \in R^{n \times n}$ 定义为除了 (i, j) 位置为 1 之外其余位置全为 0 的矩阵。换一种写法就是 $J_{n,m}^{i,j} = [i = n][j = m]$ 。

2.1 矩阵、向量与标量形式的导数

2.1.1 一阶

下面这些一阶导数展示了导数算子的基本线性性质。

$$\frac{\partial x^T a}{\partial x} = a \qquad (x \text{---} a)^\bullet = (x)^\bullet \text{---} a = -a \qquad (69)$$

$$\frac{\partial a^T x}{\partial x} = a \qquad (a \text{---} x)^\bullet = a \text{---} (x)^\bullet = a \supset = -a \qquad (69)$$

$$\frac{\partial a^T X b}{\partial X} = ab^T \qquad (a \text{---} X \text{---} b)^\bullet = a \text{---} (X)^\bullet \text{---} b = a \supset \subset b \qquad (70)$$

$$\begin{aligned}
&= (-B - + - B^T -) - x \\
&= x \begin{pmatrix} \frac{j}{i} B \frac{i}{j} \\ + \frac{j}{i} B \frac{i}{j} \end{pmatrix}
\end{aligned}$$

TODO: 假设 W 是对称的, 则…… (84) - (88)

2.1.3 高阶

像 X^n 这样的矩阵整数幂, 可以通过展开乘积并使用乘积法则来轻松处理。《Matrix Cookbook》中包含一些可以这样处理的导数。

$$\frac{\partial (\mathbf{X}^n)_{kl}}{\partial X_{ij}} = \sum_{r=0}^{n-1} (\mathbf{X}^r \mathbf{J}^{ij} \mathbf{X}^{n-1-r})_{kl} \quad \text{---} (X - X - X - \dots - X) \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix} \quad (90)$$

$$= \text{---} (X) \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix} X - X - \dots - X \text{---}$$

$$+ \dots$$

$$+ \text{---} X - X - X - \dots - (X) \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix}$$

$$= \sum_{r=0}^{n-1} \text{---} X^r \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix} X^{n-r-1} \text{---}$$

$$\frac{\partial}{\partial \mathbf{X}} \mathbf{a}^T \mathbf{X}^n \mathbf{b} = \sum_{r=0}^{n-1} (\mathbf{X}^r)^T \mathbf{a} \mathbf{b}^T (\mathbf{X}^{n-1-r})^T \quad (a - X - \dots - X - b) \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix} \quad (91)$$

$$= \sum_{r=0}^{n-1} a - X^r \begin{matrix} i \\ \diagup \\ \bullet \\ \diagdown \\ j \end{matrix} X^{n-r-1} - b$$

$$= \sum_{r=0}^{n-1} -(X^r)^T - a \quad b - (X^{n-r-1})^T -$$

2.2 迹的导数

《Matrix Cookbook》中包含大量关于迹的导数。这些式子在经典记号中可以很优雅, 因为迹是标量, 所以导数阶数较低。

2.2.1 一阶

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}(X) = I \quad & \left(\overbrace{X} \right) \nearrow = \overbrace{(X)} \nearrow \\ & = \curvearrowright \\ & = -\circ- \end{aligned} \quad (99)$$

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}(XA) = A^T \quad & \left(\overbrace{X-A} \right) \nearrow = \overbrace{(X)-A} \nearrow \\ & = \curvearrowright \overleftarrow{A} \\ & = -A^T - \end{aligned} \quad (100)$$

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}(AXB) = A^T B^T \quad & \left(\overbrace{A-X-B} \right) \nearrow = \overbrace{A-(X)-B} \nearrow \\ & = \overbrace{A \rightarrow} \overleftarrow{B} \\ & = -A^T - B^T - \end{aligned} \quad (101)$$

后续还有 (102-105)。最后一个式子使用 Kronecker 积，我们可能需要先介绍它。

2.2.2 二阶

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}(X^2) = 2X^T \quad & \left(\overbrace{X-X} \right) \nearrow \\ & = \overbrace{(X)-X} \nearrow + \overbrace{X-(X)} \nearrow \\ & = \curvearrowright \overleftarrow{X} + \overbrace{X \rightarrow} \curvearrowright \\ & = 2 - X^T - \end{aligned} \quad (106)$$

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}(X^2 B) = (XB + BX)^T \quad & \left(\overbrace{X-X-B} \right) \nearrow \\ & = \overbrace{(X)-X-B} \nearrow + \overbrace{X-(X)-B} \nearrow \\ & = \curvearrowright \overleftarrow{X-B} + \overbrace{X \rightarrow} \overleftarrow{B} \\ & = -B^T - X^T - + -X^T - B^T - \end{aligned} \quad (107)$$

$$\frac{\partial}{\partial X} \text{Tr}(X^T B X) = \frac{\partial}{\partial X} \text{Tr}(X X^T B) \quad \left(\overbrace{X^T-B-X} \right) \nearrow \quad (108, 109, 110)$$

$$\begin{aligned}
&= \frac{\partial}{\partial X} \text{Tr}(BXX^T) = \overbrace{(X^T \bullet - B - X)} + \overbrace{X^T - B - (X \bullet)} \\
&= (B + B^T)X = \overbrace{-B - X} + \overbrace{X^T - B} \\
&= -B - X - + -B^T - X - \\
\\
&\frac{\partial}{\partial X} \text{Tr}(XBX^T) = \frac{\partial}{\partial X} \text{Tr}(X^T X B) \quad (\overbrace{X - B - X^T} \bullet) \quad (111, 112, 113) \\
&= \frac{\partial}{\partial X} \text{Tr}(BX^T X) = \overbrace{(X \bullet - B - X^T)} + \overbrace{X - B - (X^T \bullet)} \\
&= X(B^T + B) = \overbrace{X - B - X^T} + \overbrace{X - B -} \\
&= -X - B^T - + -X - B -
\end{aligned}$$

最后一个方程有些出人意料, 因为我们可能会以为只要在上一个方程中把 X^T 简单替换为 X , 就能推出

$$(B + B^T)X = \frac{\partial}{\partial X} \text{Tr}(XBX^T) = \frac{\partial}{\partial X} \text{Tr}(X^T BX) = X(B^T + B).$$

然而这显然并非如此。这种替换只对线性函数有效, 对二次函数无效。一般来说, 有 $\frac{\partial}{\partial X} f(X)^T \neq \frac{\partial}{\partial X} f(X^T)$.

2.2.3 高阶

$$\begin{aligned}
&\frac{\partial}{\partial X} \text{Tr}(X^n) = n(X^{n-1})^T \quad \overbrace{(X - X - X - \dots - X)} \bullet \quad (121) \\
&= \sum_{r=0}^{n-1} \overbrace{X^r \succleftarrow X^{n-r-1}} \\
&= n(X^T)^{n-1}
\end{aligned}$$

$$\begin{aligned}
&\frac{\partial}{\partial X} \text{Tr}(AX^n) = \sum_{r=0}^{n-1} (X^r A X^{n-1-r})^T \quad \overbrace{(A - X - X - \dots - X)} \bullet \quad (122) \\
&= \sum_{r=0}^{n-1} \overbrace{A - X^r \succleftarrow X^{n-r-1}} \\
&= \sum_{r=0}^{n-1} -(X^r)^T - A^T - (X^{n-r-1})^T -
\end{aligned}$$

2.3 练习

练习 4. 求 $(x^T Ax)^2$ 关于 x 的导数。

练习 5. 求

$$x^T A^T x x^T A x$$

关于 x 的二阶导数。

练习 6. 求 $X^T A X$ 关于 X 的导数。

练习 7. 证明以下导数：

$$\frac{\partial}{\partial x} (Bx + b)^T C (Dx + d) = B^T C (Dx + d) + D^T C^T (Bx + b) \quad (78)$$

$$\frac{\partial}{\partial X} b^T X^T D X c = D^T X b c^T + D X c b^T \quad (82)$$

$$\frac{\partial}{\partial X} (Xb + c)^T D (Xb + c) = (D + D^T)(Xb + c)b^T \quad (83)$$

练习 8. 证明《Matrix Cookbook》中剩余的二阶迹导数：

$$\frac{\partial}{\partial X} \text{Tr}(A X B X) = A^T X^T B^T + B^T X^T A^T \quad (114)$$

$$\frac{\partial}{\partial X} \text{Tr}(X^T X) = \frac{\partial}{\partial X} \text{Tr}(X X^T) = 2X \quad (115)$$

$$\frac{\partial}{\partial X} \text{Tr}(B^T X^T C X B) = C^T X B B^T + C X B B^T \quad (116)$$

$$\frac{\partial}{\partial X} \text{Tr}[X^T B X C] = B X C + B^T X C^T \quad (117)$$

$$\frac{\partial}{\partial X} \text{Tr}(A X B X^T C) = A^T C^T X B^T + C A X B \quad (118)$$

$$\frac{\partial}{\partial X} \text{Tr}[(A X B + C)(A X B + C)^T] = 2A^T(A X B + C)B^T \quad (119)$$

练习 9. 证明《Matrix Cookbook》中的四阶迹导数：

$$\begin{aligned} \frac{\partial}{\partial X} \text{Tr}[B^T X^T C X X^T C X B] &= C X X^T C X B B^T \\ &\quad + C^T X B B^T X^T C^T X \\ &\quad + C X B B^T X^T C X \\ &\quad + C^T X X^T C^T X B B^T \end{aligned} \quad (123)$$

第三章 Kronecker 积与 Vec 算子

3.1 展平

展平 (flattening) 是程序员常用的操作。用 numpy 的语言来说, 我们可以写 `np.ones((2,3,4)).reshape(2, 12)`, 把形状为 (2,3,4) 的张量展平成形状为 (2,12) 的矩阵。类似地, 在数学记号中, $\text{vec}(X)$ 通常用来表示把矩阵展平成向量。

通常这么做的主要原因，是为了绕开张量通用记号不够好用的问题。因此，有了张量图，我们完全可以避免这种操作。不过，看看张量图如何让展平的许多性质变得更加透明，仍然很有意思。

首先注意，展平是一个线性操作，因此可以表示成一个简单的张量。我们用三角形表示它：

$$\triangleright_{i,j,k} = \begin{array}{c} \text{ } \diagup \text{ } i \\ \text{ } \diagdown \text{ } j \end{array} \begin{array}{c} \text{ } \diagup \text{ } k \\ \text{ } \diagdown \end{array} = [i + jn = k].$$

这里 n 是 i 边的维度。注意，我们用双线表示展平操作的输出。这只是一个记法选择，用来提醒自己：输出是一束由两条边组成的边。

用这个记号可以写成

$$\text{vec}(X)_k = \sum_{i,j} \triangleright_{i,j,k} X_{i,j} = \text{X} \begin{array}{c} \frown \\ \smile \end{array} \rhd^{\text{k}}.$$

▷ 的一些基本性质如下:

[illegible]

$$\text{Diagram 1} = \text{Diagram 2} = \text{Diagram 3} \quad (3.2)$$

$$\begin{array}{c} \text{---} \bigcirc \text{---} \\ \text{---} \bigcirc \text{---} \end{array} = \begin{array}{c} \text{---} \bigtriangleup \text{---} \\ \text{---} \bigtriangleup \text{---} \end{array} \quad (3.3)$$

$$\text{Diagram 1} = \text{Diagram 2} \quad (3.4)$$

3.2 Kronecker 积

一个 $m \times n$ 矩阵 A 与一个 $r \times q$ 矩阵 B 的 Kronecker 积, 是如下定义的 $mr \times nq$ 矩阵 $A \otimes B$:

$$A \otimes B = \begin{bmatrix} A_{1,1}B & A_{1,2}B & \cdots & A_{1,n}B \\ A_{2,1}B & A_{2,2}B & \cdots & A_{2,n}B \\ \vdots & \vdots & \ddots & \vdots \\ A_{m,1}B & A_{m,2}B & \cdots & A_{m,n}B \end{bmatrix}.$$

用指标记号也可以把它写成 $(A \otimes B)_{p(r-1)+v, q(s-1)+w} = A_{rs}B_{vw}$, 但这很难读。

在张量记号中, Kronecker 积只是两个矩阵的外积, 并且在“两侧”都做展平:

$$A \otimes B = \begin{array}{c} \text{A} \\ \text{B} \end{array}.$$

Kronecker 积有如下性质:

$$A \otimes (B + C) = A \otimes B + A \otimes C \quad \begin{array}{c} \text{A} \\ \text{(B+C)} \end{array} = \begin{array}{c} \text{A} \\ \text{B} \end{array} + \begin{array}{c} \text{A} \\ \text{C} \end{array} \quad (506)$$

$$A \otimes (B \otimes C) = (A \otimes B) \otimes C \quad \begin{array}{c} \text{A} \\ \text{B} \\ \text{C} \end{array} = \begin{array}{c} \text{A} \\ \text{B} \\ \text{C} \end{array} = \begin{array}{c} \text{A} \\ \text{B} \\ \text{C} \end{array} \quad (508)$$

$$aA \otimes bB = ab(A \otimes B) \quad \begin{array}{c} a \text{ A} \\ b \text{ B} \end{array} = \begin{array}{c} ab \text{ A} \\ \text{B} \end{array} \quad (509)$$

$$(A \otimes B)^T = A^T \otimes B^T \quad \begin{array}{c} \text{A} \\ \text{B} \end{array} = \begin{array}{c} \text{A} \\ \text{B} \end{array} \quad (510)$$

$$(A \otimes B)(C \otimes D) = AC \otimes BD \quad \begin{array}{c} \text{A} \\ \text{B} \end{array} \begin{array}{c} \text{C} \\ \text{D} \end{array} = \begin{array}{c} \text{A} \cdot \text{C} \\ \text{B} \cdot \text{D} \end{array} \quad (511)$$

$$(A \otimes I)(I \otimes B) = A \otimes B \quad \begin{array}{c} \text{A} \\ \text{B} \end{array} = \begin{array}{c} \text{A} \\ \text{B} \end{array} \quad (511b)$$

$$\text{Tr}(A \otimes B) = \text{Tr}(A)\text{Tr}(B)$$

$$\begin{aligned} \text{Diagram 1} &= \text{Diagram 2} \\ &= \text{Diagram 3} \end{aligned} \quad (515)$$

回忆矩阵的特征分解 (1.1), 还可以得到:

$$\begin{aligned} \text{eig}(A \otimes B) &= \text{eig}(A)\text{eig}(B) \\ &= \text{Diagram 1} \\ &= \text{Diagram 2} \\ &= \text{Diagram 3} \end{aligned} \tag{519}$$

其中最后一步来自 (3.4)。

如果考虑 $V = \frac{\circ}{\text{M}} \frac{\circ}{\circ}$ 表示满足 $V_{i,j,i,j} = M_{i,j}$ 、其余分量为 0 的张量，这一点会更容易看清。因此，对 V 的两侧做展平就等同于 $\text{diag}(\text{vec}(M))$ 。

3.3 Vec 算子

Vec 算子作用在矩阵 A 上时，会把各列堆叠成一个向量；也就是说，对于一个 2×2 矩阵，

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad \text{vec}(A) = \begin{bmatrix} A_{11} \\ A_{21} \\ A_{12} \\ A_{22} \end{bmatrix}$$

在本章开头,我们已经展示了如何用展平张量表示 Vec 算子: $\text{vec}(X) = X \begin{array}{c} \diagdown \\ \diagup \end{array}$. The Matrix Cookbook 给出了 Vec 算子的如下性质:

$$\text{vec}(A^T X B) = \text{vec}(X)^T (A \otimes B) \quad \text{X} \begin{array}{c} \text{A} \\ \diagup \quad \diagdown \\ \text{B} \end{array} \text{ } \text{ } = \text{X} \text{ } \text{ } \begin{array}{c} \text{A} \\ \diagdown \quad \diagup \\ \text{B} \end{array} \text{ } \text{ } \quad (520)$$

$$\text{Tr}(A^T B) = \text{vec}(A)^T \text{vec}(B) \quad A \frown B = A \rhd \lhd B \quad (521)$$

$$\text{vec}(A + B) = \text{vec}(A) + \text{vec}(B) \quad (A+B) \rhd = A \rhd + B \rhd \quad (522)$$

$$\text{vec}(aA) = a \text{vec}(A) \quad aA \rhd = a A \rhd \quad (523)$$

$$a^T X B X^T c = \text{vec}(X)^T (B \otimes c a^T) \text{vec}(X) \quad a-X-B-X-c = X \begin{matrix} \nearrow B \searrow \\ a \quad b \end{matrix} X \quad (524)$$

$$= X \rhd \lhd \begin{matrix} \nearrow B \searrow \\ a \quad b \end{matrix} X$$

3.4 Kronecker 向量积

有时, 把两个向量之间的 Kronecker 积写成矩阵会更方便。定义

$$A \otimes v = \begin{matrix} A \\ \swarrow \searrow \\ v \end{matrix} \quad (3.5)$$

$$A \otimes v^T = \begin{matrix} -A \\ \swarrow \searrow \\ v \end{matrix} \quad (3.6)$$

$$v \otimes A = \begin{matrix} v \\ \swarrow \searrow \\ A \end{matrix} \quad (3.7)$$

$$v^T \otimes A = \begin{matrix} v \\ \swarrow \searrow \\ -A \end{matrix} \quad (3.8)$$

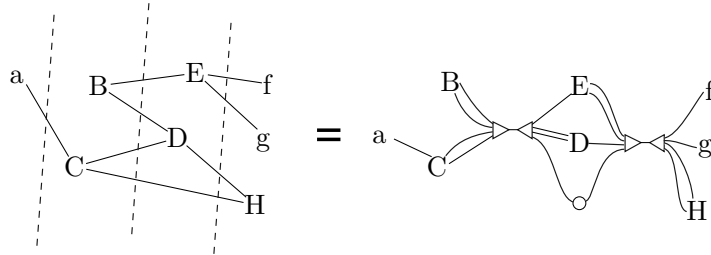
类似地, 我们可以定义两个向量之间的 Kronecker 积:

$$v \otimes w = \begin{matrix} v \\ \swarrow \searrow \\ w \end{matrix} \quad (3.9)$$

$$v^T \otimes w^T = \begin{matrix} v \\ \swarrow \searrow \\ w \end{matrix} \quad (3.10)$$

3.5 一般矩阵化

最后一个方程体现了一个一般思想: 对所有张量应用 Vec 算子、对所有边应用展平张量, 就可以把任意张量网络转换成一串矩阵乘法。例如, 下面这个复杂的图:



可以写成一个简单的向量-矩阵-矩阵-向量乘积 aM_1M_2b , 其中 $M_1 = \text{vec}(B) \otimes C'$, $M_2 = E' \otimes D' \otimes I$ 且 $b = f \otimes g \otimes \text{vec}(H)$ 。这里 C' 、 D' 和 E' 是在一侧展平的三阶张量, 而 $\text{vec}(B)$ 被解释为只有一列的矩阵。

3.5.1 Lyapunov 方程

Kronecker 积重写的一个漂亮应用, 是求解如下方程:

$$AX + XB = C. \quad (272)$$

我们使用重写 $\text{vec}(AX + XB) = (I \otimes A + B^T \otimes I)\text{vec}(X)$, 它来自如下张量图变形:

$$\left(\begin{array}{c} -A-X- \\ + \quad -X-B- \end{array} \right) \curvearrowright = X \overset{A}{\curvearrowright} + X \underset{B}{\curvearrowright} = X \curvearrowright \left(\begin{array}{c} \curvearrowleft A \curvearrowright \\ \curvearrowleft B \curvearrowright \end{array} \right) =$$

之后取通常意义下的矩阵逆, 就得到

$$\text{vec}(X) = (I \otimes A + B^T \otimes I)^{-1} \text{vec}(C). \quad (273)$$

3.5.2 封装求和

这是前一个方程的推广。

$$\sum_n A_n X B_n = C \quad (274)$$

$$\text{vec}(X) = \left(\sum_n B_n^T \otimes A_n \right)^{-1} \text{vec}(C) \quad (275)$$

3.6 Hadamard 积

Hadamard 积也称为逐元素乘法, The Matrix Cookbook 中没有介绍它。不过, 它是一个非常有用的操作, 并且有一些与 Kronecker 积相关的有趣性质。

两个 2×2 矩阵 A 和 B 的 Hadamard 积定义为

$$A \circ B = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \circ \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} & A_{12}B_{12} \\ A_{21}B_{21} & A_{22}B_{22} \end{bmatrix}.$$

在张量记号中, Hadamard 积可以用两个三阶复制张量表示:

$$A \circ B = - \circ \begin{array}{c} \text{A} \\ \text{B} \end{array} \circ -.$$

Hadamard 积的一些性质如下:

$$\begin{aligned} x^T(A \circ B)y &= \text{tr}(A^T D_x B D_y) & x - \circ \begin{array}{c} \text{A} \\ \text{B} \end{array} \circ - y &= \overbrace{A^T - \circ - B - \circ}^{\substack{x \\ y}} \\ (A \otimes B) \circ (C \otimes D) &= (A \circ C) \otimes (B \circ D) & \text{Diagram 1} &= \text{Diagram 2} \end{aligned}$$

第一个方程只是对张量图做变形。第二个方程来自 (3.3)。另一种看法是, 只要沿着双线追踪即可: A 和 C 都使用双边的“上”半部分, 而 B 和 D 使用下半部分。

3.7 Khatri-Rao 积

它也称为按列 Kronecker、按行 Kronecker 或“Face-splitting Product”。我们分别用符号 $*$ 和 $\bullet$ 表示按列与按行 Kronecker 积。

$$\begin{aligned} A * B &= \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} * \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} A_{11}B_{11} & A_{12}B_{12} \\ A_{11}B_{21} & A_{12}B_{22} \\ A_{21}B_{11} & A_{22}B_{12} \\ A_{21}B_{21} & A_{22}B_{22} \end{bmatrix} \\ A \bullet B &= \dots = \begin{bmatrix} A_{11}B_{11} & A_{11}B_{12} & A_{12}B_{11} & A_{12}B_{12} \\ A_{21}B_{21} & A_{21}B_{22} & A_{22}B_{21} & A_{22}B_{22} \end{bmatrix} \end{aligned}$$

用张量图来说, 这些积只是在乘积的一侧做展平, 并在另一侧使用复制张量:

$$\begin{aligned} A * B &= \begin{array}{c} \text{A} \\ \text{B} \end{array} \circ - \\ A \bullet B &= - \circ \begin{array}{c} \text{A} \\ \text{B} \end{array} \end{aligned}$$

显然，二者只差一个转置。事实上， $(A * B)^T = B^T \bullet A^T$ 且 $(A \bullet B)^T = B^T * A^T$ 。

存在多个“混合乘积”恒等式：

$$(A \bullet B)(C \otimes D) = (AC) \bullet (BD) \quad - \circ \begin{array}{c} A \\ B \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} \begin{array}{c} C \\ D \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} = - \circ \begin{array}{c} A \cdot C \\ B \cdot D \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array}$$

$$(Ax) \circ (By) = (A \bullet C)(x \otimes y) \quad - \circ \begin{array}{c} A \cdot x \\ B \cdot y \end{array} = - \circ \begin{array}{c} A \\ B \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} \begin{array}{c} x \\ y \end{array}$$

3.7.1 拼接

如果两个张量的形状匹配，就可以沿某个轴把它们拼接起来。一种简单做法是，分别与一个选择新轴中位置的向量取外积，然后把结果相加。定义 $e^{(i)}$ 为长度为 2 的向量，它在第 i 个位置为 1，其余位置为 0。也就是说， $e^{(0)} = [1, 0]$ 且 $e^{(1)} = [0, 1]$ 。于是可以定义

$$\text{cat}_i(X, Y) = \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(0)} X_{\bar{j}} + \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(1)} Y_{\bar{j}}.$$

对于矩阵和向量，我们常用竖线来表示拼接，例如： $\text{stack}_i(A, B) = A \mid B$ 和 $\text{stack}_j(A, B) = A \mid B$ 。由此得到恒等式

$$\begin{aligned} (A \mid C) \begin{pmatrix} B \\ D \end{pmatrix} &= \text{stack}_i(A, C) \text{stack}_i(B, D) \\ &= \left(\begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(0)} A_{\bar{j}} + \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(1)} C_{\bar{j}} \right) \left(\begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(0)} B_{\bar{m}} + \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(1)} D_{\bar{m}} \right) \\ &= \frac{e^{(0)} - e^{(0)}}{j} A \text{---} B_{\bar{m}} + \frac{e^{(0)} - e^{(1)}}{j} A \text{---} D_{\bar{m}} + \frac{e^{(1)} - e^{(1)}}{j} C \text{---} D_{\bar{m}} + \frac{e^{(1)} - e^{(0)}}{j} C \text{---} B_{\bar{m}} \\ &= AB + CD, \end{aligned}$$

因为 $e^{(0)} - e^{(0)} = e^{(1)} - e^{(1)} = 1$ ，且 $e^{(0)} - e^{(1)} = 0$ 。

堆叠操作与矩阵的直和密切相关。两个矩阵的直和 $A \oplus B$ 会产生一个块对角矩阵：

$$A \oplus B = \begin{bmatrix} A & 0 \\ 0 & B \end{bmatrix}$$

在张量图记号中，可以用类似堆叠的构造来表示直和：

$$A \oplus B = \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(0)} \begin{array}{c} e^{(0)} \\ A \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(1)} + \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(1)} \begin{array}{c} e^{(1)} \\ B \end{array} \begin{array}{c} \diagup \diagdown \\ \diagdown \diagup \end{array} e^{(0)}$$

这个构造清楚地表明，直和会把 A 放在左上块（当 $i = j = 0$ 时），把 B 放在右下块（当 $i = j = 1$ 时），其他位置为零。这种联系在量子计算中尤其重要，在那里这个操作称为“控制”（controlling）：根据控制量子比特的状态应用不同操作。

更一般地, 对多个矩阵有:

$$\bigoplus_k A_k = \sum_k \text{diag}(e^{(k)} A_k e^{(k)})$$

其中 $e^{(k)}$ 定义为长度为 n 的向量, 它在第 k 个位置为 1, 其余位置为 0, $k = 0, 1, \dots, n-1$ 。

直和相对于 Kronecker 积满足若干性质:

$$\begin{aligned} A \otimes (B \oplus C) &= \text{diag}(A \otimes B, A \otimes C) \\ &= (A \otimes B) \oplus (A \otimes C) \\ (A \oplus B)(C \oplus D) &= \begin{pmatrix} A & 0 \\ 0 & B \end{pmatrix} \begin{pmatrix} C & 0 \\ 0 & D \end{pmatrix} \\ &= AC \oplus BD \end{aligned} \quad \begin{array}{l} \text{(yyy)} \\ \text{(xxx)} \end{array}$$

3.8 导数

3.9 练习

练习 10. 设 $J = \text{Tr}[(I_N \otimes F)^T A (I_N \otimes F) B]$, 其中 $F \in \mathbb{R}^{N \times Nn}$, $A \in \mathbb{R}^{N^2 \times N^2}$, $B \in \mathbb{R}^{N^2 n \times N^2 n}$. 求 J 关于 F 的导数。

练习 11. 考虑 $J = \|G - (B \otimes X)\|_F^2$, 其中 G 和 B 是矩阵, $\|\cdot\|_F$ 是 Frobenius 范数。求它关于 X 的导数。

练习 12. 证明方程 [8]:

$$\text{vec}(A \text{diag}(b) C) = ((C^T \otimes 1) \circ (1 \otimes A))b.$$

这里 $\text{diag}(b)$ 是以向量 b 为对角线的对角矩阵, 1 是尺寸合适的全 1 向量, 而 $b \otimes A$, 即向量与矩阵的 Kronecker 积, 定义为 $\text{diag}_A(b)$, 也就是说, 只在一侧做展平。

练习 13. 设 a 和 b 是两个向量, D 和 X 是两个矩阵. 关于 X 最小化如下代价函数:

$$E = \|a - DXb\|_2^2.$$

练习 14. 用图证明 $\text{Tr}(A^T B) = 1^T \text{vec}(A \circ B)$, 其中 1 是尺寸合适的向量.

练习 15. 求

$$\text{Tr}(G(A \otimes X))$$

关于 X 的导数.

练习 16.

$$\frac{\partial}{\partial \mathbf{X}} \text{Tr}(\mathbf{X} \otimes \mathbf{X}) = \frac{\partial}{\partial \mathbf{X}} \text{Tr}(\mathbf{X}) \text{Tr}(\mathbf{X}) = 2 \text{Tr}(\mathbf{X}) \mathbf{I}$$

练习 17. 求

$$\text{Tr}(G(A \otimes X))$$

关于 X 的导数, 其中 G 是一个 $\mathbb{R}^{n^2} \times \mathbb{R}^{n^2}$ 矩阵, 而 A 和 X 是 $\mathbb{R}^{n \times n}$ 矩阵.

用 $\text{vec}(A)$ 和 G 的某个重塑版本来写出结果.

练习 18. 验证如下恒等式:

$$\frac{d}{dx} \text{vec}(\text{diag}(x) A \text{diag}(x)) = \text{diag}(\text{vec}(A))(I \otimes x + x \otimes I).$$

提示: 使用矩阵-向量恒等式 (3.5).

练习 19. 证明

$$\frac{\partial}{\partial x} x^T A (x \otimes x) = A(x \otimes x) + (x^T \otimes I + I \otimes x^T) A^T x$$

提示: 使用矩阵-向量恒等式 (3.5).

第四章 函数

本章讨论从张量到其他张量的一般函数。它们虽然不如线性函数优雅，却对许多应用都很重要，例如神经网络中的非线性。主要目标是让向量链式法则变得直观且易于应用，不过我们也会讨论 Taylor 级数、行列式和其他主题。

当函数会沿某些维度广播时，向量函数的标准记号可能相当含糊。如果 $x \in \mathbb{R}^n$ 是一个向量，那么 $f(x)$ 到底是独立作用在 x 的每个元素上，还是把整个向量作为输入，并不清楚。借助张量图，我们可以显式画出 x 的哪些边进入 f 、哪些边不进入，从而澄清这种区别。我们使用记号 ${}^m f \leftarrow^n x$ 表示 f 是从 $\mathbb{R}^n$ 到 $\mathbb{R}^m$ 的函数。如果 f 是逐元素函数，则写作 $(f \leftarrow^n x) \in \mathbb{R}^n$ 。我们总是假设函数边（箭头边）先于普通边缩并。如果想表达别的内容，例如 $f(Mx)$ ，可以使用括号： $-f \leftarrow (-M - x)$ 。再看一些例子会更有帮助：

$$f: \mathbb{R}^n \rightarrow \mathbb{R} \quad f(x) \in \mathbb{R} \quad f \stackrel{n}{\leftarrow} x \quad (\text{标量函数})$$

$$g: \mathbb{R}^n \rightarrow \mathbb{R}^m \quad g(x) \in \mathbb{R}^m \quad \stackrel{m}{\leftarrow} g \stackrel{n}{\leftarrow} x \quad (\text{向量函数})$$

$$h: \mathbb{R} \rightarrow \mathbb{R} \quad h(x) \in \mathbb{R}^n \quad h \stackrel{\leftarrow \cdots}{\leftarrow} x \stackrel{\leftarrow \cdots}{\leftarrow} n \quad (\text{逐元素函数})$$

$$\begin{array}{l} u: \mathbb{R}^n \rightarrow \mathbb{R}^m \\ v \in \mathbb{R}^m \end{array} \quad v^T u(x) \in \mathbb{R} \quad u \stackrel{m}{\leftarrow} v \stackrel{n}{\leftarrow} x \quad (\text{向量乘向量函数})$$

$$\begin{array}{l} A: \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}, \\ v \in \mathbb{R}^n \end{array} \quad A(x)v \in \mathbb{R}^m \quad \stackrel{m}{\leftarrow} A \stackrel{n}{\leftarrow} x \quad (\text{向量乘矩阵函数})$$

$$\begin{array}{l} f: \mathbb{R}^d \rightarrow \mathbb{R}, \\ X \in \mathbb{R}^{b \times d} \end{array} \quad f(X) \in \mathbb{R}^b \quad f \stackrel{d}{\leftarrow} X \stackrel{b}{\leftarrow} \quad (\text{向量函数, 批量形式})$$

$$\begin{array}{l} A: \mathbb{R}^{n \times m} \rightarrow \mathbb{R}, \\ X \in \mathbb{R}^{n \times m} \end{array} \quad A(X) \in \mathbb{R} \quad A \stackrel{n}{\leftarrow} \stackrel{m}{\leftarrow} X \quad (\text{矩阵输入函数})$$

$$\begin{array}{l} f: \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^d, \\ u \in \mathbb{R}^{b \times n}, v \in \mathbb{R}^{b \times m} \end{array} \quad f(u, v) \in \mathbb{R}^{b \times d} \quad \stackrel{d}{\leftarrow} f \stackrel{n}{\leftarrow} \stackrel{u}{\leftarrow} \stackrel{b}{\leftarrow} \stackrel{m}{\leftarrow} v \stackrel{b}{\leftarrow} \quad (\text{两个输入, 批量形式})$$

为了更具体一些, 考虑几个熟悉的函数: (1) 行列式函数 $\det: \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$ 是一个标量函数, 我们可以将它应用到一批矩阵上: $\det \stackrel{n}{\leftarrow} \stackrel{m}{\leftarrow} X \stackrel{b}{\leftarrow}$, 结果得到一个向量 $\in \mathbb{R}^b$. (2) 余弦相似度 $\text{cossim}: \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是一个标量函数, 可以把它应用到两个向量上: $\text{cossim} \stackrel{n}{\leftarrow} \stackrel{u}{\leftarrow} \stackrel{v}{\leftarrow}$, 结果得到一个标量 $\in \mathbb{R}$. (3) 如果逐元素幂函数 $\text{pow}_n: \mathbb{R} \rightarrow \mathbb{R}$ 应用于两个向量, 那么它们在 Hadamard 积下可以化简:

$$\frac{\text{pow}_n \stackrel{\leftarrow \cdots}{\leftarrow} u}{\text{pow}_m \stackrel{\leftarrow \cdots}{\leftarrow} u} \stackrel{\circ}{\leftarrow} = \text{pow}_{n+m} \stackrel{\leftarrow \cdots}{\leftarrow} u \stackrel{\leftarrow \cdots}{\leftarrow}$$

作为一个更进阶的例子, 考虑 softmax 函数: $\stackrel{d}{\leftarrow} \text{softmax} \stackrel{d}{\leftarrow} x$. 可以把它写成如下初等函数组合:

$$\text{softmax}(x) = \frac{\exp(x)}{\text{sum}(\exp(x))} = \frac{\exp \stackrel{\leftarrow \cdots}{\leftarrow} x \stackrel{\leftarrow \cdots}{\leftarrow}}{\text{pow}_{-1} \stackrel{\leftarrow \cdots}{\leftarrow} (\exp \stackrel{\leftarrow \cdots}{\leftarrow} x \stackrel{\leftarrow \cdots}{\leftarrow} \circ)}$$

一开始它看起来有点复杂, 但我们可以逐步拆开: $(\exp \stackrel{\leftarrow \cdots}{\leftarrow} x \stackrel{\leftarrow \cdots}{\leftarrow}) \in \mathbb{R}^n$ 是逐元素指数

函数。如果把它与 $- \circ$ 缩并, 就得到和 $s = \sum_i \exp(x_i)$ 。最后, 将 pow_{-1} 作用到 s 上, 并与指数函数相乘, 就得到 softmax 函数。

有一种情况可能会稍显混乱: 输出张量与输入张量的广播边发生缩并。但只要记住先缩并函数边, 一切就仍然成立。例如, 对于函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 和矩阵 $x \in \mathbb{R}^{m \times n}$:

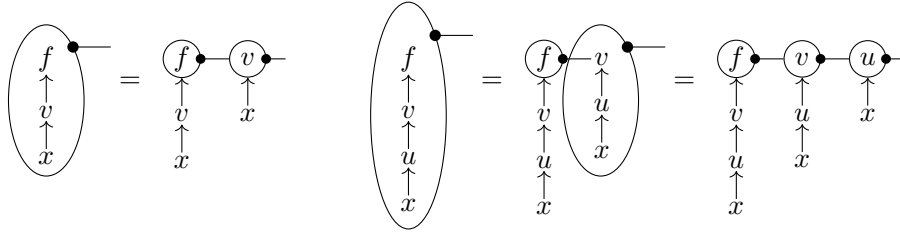
$$f \overleftarrow{\underbrace{\quad}_n^m} x = \overbrace{f(x)}^m = \text{Tr}(f(x)).$$

4.1 链式法则

张量图最吸引人的特性之一, 是链式法则在其中非常透明。对于两个函数 $f: \mathbb{R}^m \rightarrow \mathbb{R}$ 和 $v: \mathbb{R}^n \rightarrow \mathbb{R}^m$, 复合函数 $f \circ v: \mathbb{R}^n \rightarrow \mathbb{R}$ 只需把 v 的输出送入 f , 就可以画成图:

$$f \leftarrow v \leftarrow x$$

在传统记号中, 链式法则写作 $J_{f \circ v}(x) = \nabla f(v(x)) J_v(x)$, 其中 $J_v(x)$ 是 v 的 Jacobian 矩阵, $\nabla f(v(x))$ 是 f 在 $v(x)$ 处的梯度。在张量图中, 链式法则真的就是一条链!



这清楚地符合经典口诀: “外层函数的导数乘以内层函数的导数”。唯一需要细化的是, “外层函数”要通过求导产生的新边连接到“内层函数”。

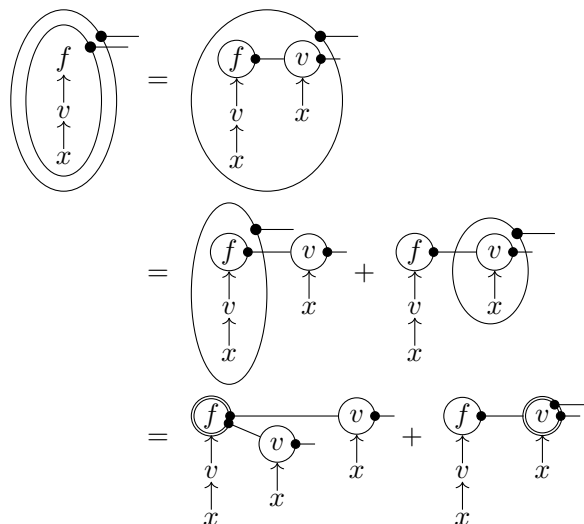
也可以进一步把 $\textcircled{f}$ 简化成 f' , 其中 f' 是 f 的导数。例如, 如果 f 是 $\cos -$, 那么 f' 就是 $\sin -$ 。不过, 也可以继续保留带圈的 f , 把它作为容易识别的导数符号。

4.1.1 Hessian 链式法则

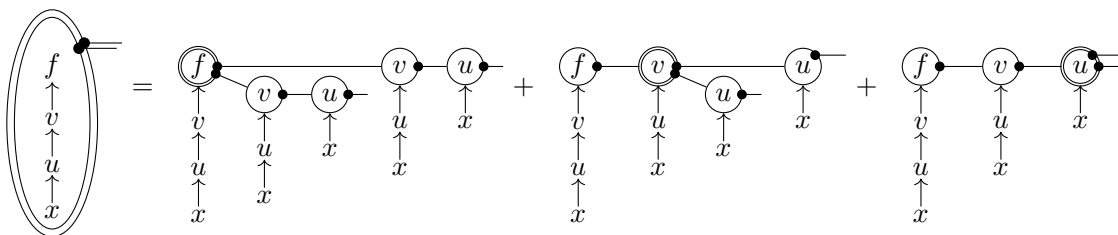
当考虑二阶导数时, 事情会变得更有趣。使用标准记号, $f \circ v$ 的 Hessian 矩阵为

$$H_{f \circ v}(x) = Dv(x)^T \cdot D^2 f(v(x)) \cdot Dv(x) + \sum_{k=1}^d \frac{\partial f}{\partial u_k}(v(x)) \frac{\partial^2 v_k}{\partial x \partial x^T}(x).$$

这个“Hessian 链式法则”用张量图推导要容易得多：



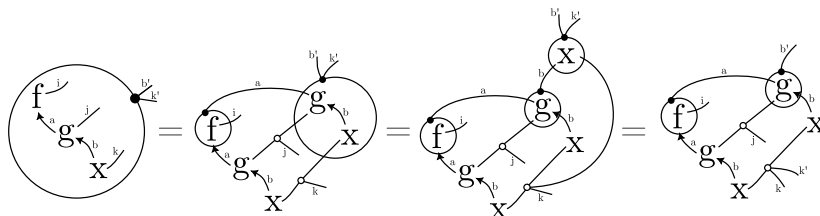
如果继续以这种方式展开，就会看到更多函数的链式法则中存在一个简单模式：



Yaroslav Bulatov 对 Hessian 链式法则以及如何在深度学习应用中高效求值写过大量文章。感兴趣的读者可以参考他关于这一主题的博客文章。

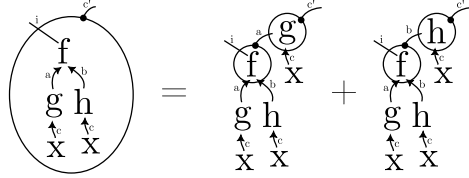
4.1.2 带广播的链式法则

当链式法则应用于带广播的函数时，结果仍然遵循“外层导数乘以内层导数”的口诀，只是“乘”的含义扩展为对广播边取 Hadamard 积。



4.1.3 多输入函数

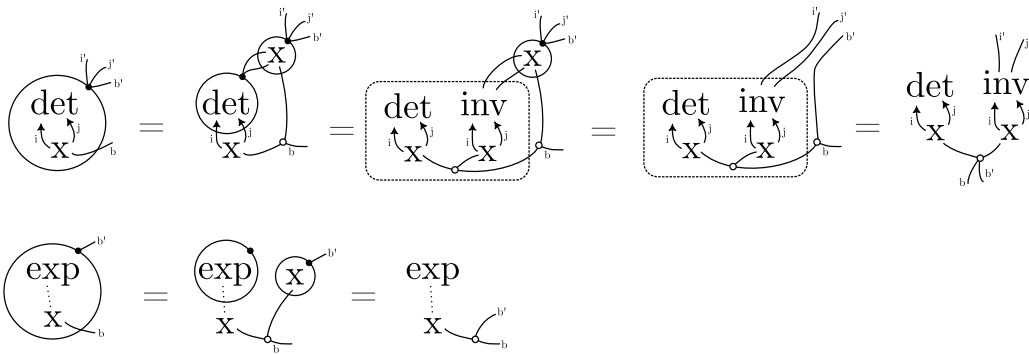
如果 $f: \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}$ 是一个有两个输入的函数, 那么 $f(x, y)$ 的导数就是 $\frac{\partial f}{\partial x} \partial x + \frac{\partial f}{\partial y} \partial y$, 用张量图记号则为:



4.2 例子

4.2.1 已知导数

下面两个例子中, 可以用已知导数展开 $(f) \bullet$:



这里 $\text{inv}(X)$ 是矩阵求逆函数, 它把矩阵 X 映到其逆矩阵 X^{-1} 。对于行列式, 可以继续求导并发现如下简单模式:

$$\frac{\partial^k \det(X)}{\partial X^k} = \det(X) (X^{-1})^{\otimes k}.$$

4.2.2 伪线性形式

伪线性形式 $A(x)x$ 很常见。所有像素自适应滤波器, 例如 non-local means、bilateral 等, 以及 transformers 中所谓的注意力机制, 都可以写成这种形式。根据 Peyman Milanfar 的说法, 这个函数的梯度很重要, 而且具有值得记住的形式:

$$\begin{aligned} \text{---} \left(\begin{array}{c} A \\ \uparrow \\ x \end{array} \right) x &= \text{---} \left(\begin{array}{c} A \\ \uparrow \\ x \end{array} \right) x + \text{---} A \left(\begin{array}{c} x \end{array} \right) = \text{---} \left(\begin{array}{c} A \\ \uparrow \\ x \end{array} \right) x + \text{---} A \end{aligned}$$

如果对比 Peyman Milanfar 用经典记号给出的如下推导, 就更能体会这个表达式的简洁:

$$\begin{aligned}
 d[A(x)x] &= d[A(x)]x + A(x)dx \\
 &= \text{vec } d[A(x)]x + A(x)dx \\
 &= \text{vec } Id[A(x)]x + A(x)dx \\
 &= (x^T \otimes I) \text{vec } d[A(x)] + A(x)dx \\
 &= (x^T \otimes I) D \text{vec}[A(x)]dx + A(x)dx \\
 &= [(x^T \otimes I) D \text{vec}[A(x)] + A(x)]dx
 \end{aligned}$$

这最终推出:

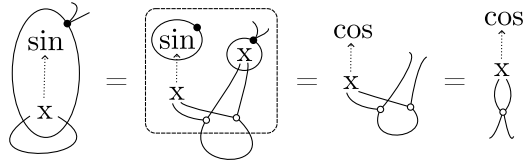
$$\frac{\partial A(x)x}{\partial x} = (x^T \otimes I) \frac{\partial}{\partial x} \text{vec}[A(x)] + A(x).$$

4.2.3 迹恒等式

The Matrix Cookbook 给出公式:

$$\frac{\partial \text{Tr}(\sin(x))}{\partial x} = \cos(x)^T \quad (128)$$

如果尝试用张量图推导, 会得到如下不同结果:



它等价于 $\cos(x) \circ I$ 。也就是说, 它是 $\cos(x)$, 但除对角线外处处为零。

原因在于 The Matrix Cookbook 实际上使用了略有不同的“函数作用于矩阵”的定义。如果 F 可以写成幂级数, 那么定义 $F(X)$ 的一种方式就是矩阵幂级数:

$$F(X) = \sum_{k=0}^{\infty} a_k X^k.$$

在这种情况下, $\text{Tr}(F(X))$ 的导数是 $f(X)^T$, 其中 $f(X)$ 是 $F(X)$ 的标量导数, 这与 The Matrix Cookbook 的公式一致。

4.2.4 Taylor

对于一个 n 次可微函数 $v: \mathbb{R}^d \rightarrow \mathbb{R}^d$, 可以写出 Taylor 展开:

$$\begin{array}{c} v \\ \uparrow \\ (x + \varepsilon) \end{array} \approx \begin{array}{c} v \\ \uparrow \\ x \end{array} + \varepsilon \begin{array}{c} \bullet \\ \uparrow \\ x \end{array} + \frac{1}{2} \varepsilon \begin{array}{c} \bullet \\ \uparrow \\ x \end{array} + \frac{1}{6} \varepsilon \begin{array}{c} \bullet \\ \uparrow \\ x \end{array} + \dots$$

用经典记号写出来会相当混乱:

$$\begin{aligned} v(x + \varepsilon) &\approx v(x) + \left[\frac{\partial}{\partial x} v(x) \right] \varepsilon + \frac{1}{2} \left[\frac{\partial}{\partial x} \left[\frac{\partial}{\partial x} v(x) \right] \varepsilon \right] \varepsilon + \frac{1}{6} \left[\frac{\partial}{\partial x} \left[\frac{\partial}{\partial x} \left[\frac{\partial}{\partial x} v(x) \right] \varepsilon \right] \varepsilon \right] \varepsilon + \dots \\ &= v(x) + \left[\frac{\partial}{\partial x} v(x) \right] \varepsilon + \frac{1}{2} (I \otimes \varepsilon) \left[\frac{\partial \text{vec}}{\partial x} \left[\frac{\partial v(x)}{\partial x} \right] \right] \varepsilon + \frac{1}{6} (I \otimes \varepsilon \otimes \varepsilon) \left[\frac{\partial \text{vec}}{\partial x} \left[\frac{\partial \text{vec}}{\partial x} \left[\frac{\partial v(x)}{\partial x} \right] \right] \right] \varepsilon + \dots \end{aligned}$$

用指标记号则还可以:

$$v_i(x + \varepsilon) \approx v_i(x) + \sum_j \frac{\partial v_i(x)}{\partial x_j} \varepsilon_j + \frac{1}{2} \sum_{j,k} \frac{\partial^2 v_i(x)}{\partial x_j \partial x_k} \varepsilon_j \varepsilon_k + \frac{1}{6} \sum_{j,k,\ell} \frac{\partial^3 v_i(x)}{\partial x_j \partial x_k \partial x_\ell} \varepsilon_j \varepsilon_k \varepsilon_\ell$$

4.3 练习

练习 20. 为函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 画出张量图: 它先应用逐元素非线性 (例如 $\exp$), 再对各分量求和。验证该图对应于 softmax 分母的常规公式。

练习 21. 用张量图表示两个函数的复合:

$$f: \mathbb{R}^m \rightarrow \mathbb{R} \quad \text{和} \quad v: \mathbb{R}^n \rightarrow \mathbb{R}^m,$$

然后使用图形式的链式法则, 推导 $f \circ v$ 关于 $x \in \mathbb{R}^n$ 的导数表达式。

练习 22. 对依赖向量 x 的矩阵函数 $A(x)$, 使用张量图说明导数

$$\frac{\partial}{\partial x} [A(x)x],$$

并解释乘积法则如何在图中体现。

练习 23. 将 Variational Autoencoder (VAE) 中的 KL 散度项表示为

$$KL(\mu, \sigma) = -\frac{1}{2} \left(1 + \log \sigma^2 - \mu^2 - \sigma^2 \right),$$

其中参数为

$$\mu = Wx + c \quad \text{且} \quad \log \sigma^2 = Wx + c.$$

推导关于权重矩阵 W 的梯度 $\nabla_W KL$ 。注意跟踪维度, 并处理逐元素操作。

练习 24. 设 softmax 函数 $s: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 定义为

$$s_i(z) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad i = 1, \dots, n.$$

证明 s 的 Jacobian 矩阵为

$$\frac{\partial s_i}{\partial z_j} = s_i(\delta_{ij} - s_j),$$

其中 δ_{ij} 是 Kronecker delta。

练习 25. 设 $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$ 是来自多元正态分布 $N(\mu, \Sigma)$ 的独立样本。写出对数似然函数，并推导梯度 $\nabla_{\mu} \ell$ 和 $\nabla_{\Sigma} \ell$ 。然后令这些梯度为零，证明最大似然估计量为

$$\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x^{(i)} \quad \text{且} \quad \hat{\Sigma} = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \hat{\mu})(x^{(i)} - \hat{\mu})^T.$$

练习 26. 考虑一个具有协方差矩阵 $K(\theta)$ 的高斯过程，其对数边际似然定义为

$$L(\theta) = -\frac{1}{2} \left(y^T K(\theta)^{-1} y + \log \det K(\theta) \right).$$

推导 L 关于 θ 的梯度，并证明

$$\frac{\partial L}{\partial \theta} = \frac{1}{2} \text{tr} \left(\left(K^{-1} y y^T K^{-1} - K^{-1} \right) \frac{\partial K}{\partial \theta} \right).$$

练习 27. 在线性逻辑回归中，设

$$p_i = \sigma(w^T x_i) \quad \text{其中} \quad \sigma(z) = \frac{1}{1 + e^{-z}},$$

并考虑负对数似然

$$J(w) = - \sum_{i=1}^n \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right].$$

推导梯度 $\nabla_w J$ 和 Hessian 矩阵 $H = \nabla_w^2 J$ 。特别地，证明

$$\nabla_w J = \sum_{i=1}^n (p_i - y_i) x_i, \quad \text{且} \quad H = \sum_{i=1}^n p_i(1 - p_i) x_i x_i^T.$$

练习 28. 设 $X \in \mathbb{R}^{n \times n}$ 是可逆矩阵。证明

$$\frac{\partial \det(X)}{\partial X} = \det(X) (X^{-1})^T.$$

换言之，证明 $\det(X)$ 关于 X 的梯度等于 $\det(X)$ 乘以 X^{-1} 的转置。

练习 29. 对可逆矩阵 $X \in \mathbb{R}^{n \times n}$, 证明

$$\frac{\partial}{\partial X} \ln \det(X) = (X^{-1})^T.$$

简要讨论为什么该结果在高斯似然等统计应用中有用。

练习 30. 设 $A \in \mathbb{R}^{n \times n}$ 是可逆矩阵 (它可能依赖某个参数)。从恒等式 $I = A^{-1}A$ 出发, 对两边求微分, 证明

$$d(A^{-1}) = -A^{-1}(dA)A^{-1}.$$

推出偏导数 $\frac{\partial A^{-1}}{\partial A_{ij}}$ 的表达式。

练习 31. 定义标量函数

$$f(A) = a^T A^{-1} b,$$

其中 $a, b \in \mathbb{R}^n$ 是固定向量, 且 $A \in \mathbb{R}^{n \times n}$ 可逆。使用矩阵求逆微分的结果, 证明

$$\nabla_A f(A) = -A^{-T}(a b^T)A^{-T}.$$

练习 32. 设 $X \in \mathbb{R}^{n \times n}$ 是对称矩阵, 具有单重特征值 λ 及其对应的单位特征向量 v 。证明 λ 关于 X 的导数为

$$\nabla_X \lambda = v v^T.$$

另外, 讨论为什么当 λ 的重数大于一时该结果不成立。

练习 33. 对于对称矩阵 $A \in \mathbb{R}^{n \times n}$, 考虑 Rayleigh 商

$$R(x) = \frac{x^T A x}{x^T x}, \quad x \in \mathbb{R}^n \setminus \{0\}.$$

使用 Lagrange 乘子, 证明 $R(x)$ 的驻值对应于 A 的特征值, 并且最大化点 (最小化点) 是与最大 (最小) 特征值对应的特征向量。

练习 34. 设 $A \in \mathbb{R}^{m \times m}$ 和 $B \in \mathbb{R}^{n \times n}$ 为常矩阵, $X \in \mathbb{R}^{m \times n}$ 为变量矩阵。证明

$$\nabla_X \operatorname{Tr}(X^T A X B) = A X B^T + A^T X B.$$

(提示: 使用迹的循环性质重新排列各项。)

练习 35. (a) 证明对于常矩阵 $A \in \mathbb{R}^{m \times n}$ 和变量 $X \in \mathbb{R}^{m \times n}$,

$$\frac{\partial}{\partial X} \operatorname{Tr}(A^T X) = A.$$

(b) 更一般地, 对常矩阵 $A \in \mathbb{R}^{p \times m}$ 和 $B \in \mathbb{R}^{n \times q}$, 证明

$$\nabla_X \operatorname{Tr}(A X B) = A^T B^T.$$

练习 36. 设 $s \in \mathbb{R}^n$ 是向量, $C \in \mathbb{R}^{n \times n}$ 是固定矩阵。定义矩阵函数

$$F(s) = \text{diag}(s) C \text{diag}(s),$$

其中 $\text{diag}(s)$ 表示以 s 的元素为对角线的对角矩阵。用 s 、 ds 和 C 表示微分 dF , 并由此推导出 $\nabla_s F(s)$ 的表达式。

练习 37. 设 $A \in \mathbb{R}^{p \times m}$ 、 $X \in \mathbb{R}^{m \times n}$ 且 $B \in \mathbb{R}^{n \times q}$ 。证明恒等式

$$\text{vec}(A X B^T) = (B \otimes A) \text{vec}(X).$$

讨论如何使用这个恒等式把某些矩阵导数问题转换成向量化形式。

练习 38. 设

$$f(x) = x^T A x,$$

其中 $x \in \mathbb{R}^n$, $A \in \mathbb{R}^{n \times n}$ 不一定对称。证明

$$\nabla_x f(x) = (A + A^T)x.$$

然后计算 Hessian 矩阵 $\nabla_x^2 f(x)$, 并讨论当 A 对称时会出现什么化简。

练习 39. 考虑 Frobenius 范数 $\|X\|_F = \sqrt{\text{Tr}(X^T X)}$, 其中 $X \in \mathbb{R}^{m \times n}$ 。

证明

$$\nabla_X \|X\|_F = \frac{X}{\|X\|_F}$$

对 $X \neq 0$ 成立。

练习 40. 设 $Y \in \mathbb{R}^{n \times n}$ 是对称矩阵, 并定义 $\phi(Y)$ 为其最大特征值。如果该特征值的重数为 $k > 1$, 证明 $\phi(Y)$ 的导数并不唯一。特别地, 说明如下形式的任意矩阵

$$G = \sum_{i=1}^k u_i v_i^T,$$

其中 $\{u_i\}_{i=1}^k$ 是最大特征值对应特征空间的任意标准正交基, 都是 $\phi(Y)$ 的有效次梯度。解释在这种情形下定义唯一梯度会遇到的困难。

第五章 统计与概率

5.1 矩的定义

设 $x \in \mathbb{R}^n$ 是一个随机变量。我们记 $m = E[x] \in \mathbb{R}^n$ 为期望，记 $M = \text{Var}[x] = E[(x - m)(x - m)^T]$ 为协方差（当这些量有定义时）。

在张量图中，我们会使用方括号：

$$-m = [-x] \quad \text{和} \quad -M- = [-(x \ominus m) \quad (x \ominus m)-]$$

我们使用带圈减号 $\ominus$ ，以便把这个操作和缩并边区分开来。

我们还可以定义三阶和四阶中心矩张量：

$$M_3 \begin{smallmatrix} \diagup \\ \diagdown \end{smallmatrix} = \begin{bmatrix} (x \ominus m)- \\ (x \ominus m)- \\ (x \ominus m)- \end{bmatrix} \quad \text{和} \quad M_4 \begin{smallmatrix} \diagup \\ \diagdown \end{smallmatrix} = \begin{bmatrix} (x \ominus m)- \\ (x \ominus m)- \\ (x \ominus m)- \\ (x \ominus m)- \end{bmatrix}.$$

5.1.1 线性组合的期望

一般原则：“期望的线性性”允许你把图中所有不涉及 X 的部分提出去。

$$\left[\begin{array}{ccccc} A & & X & - & B & & C \\ | & \diagup & | & & \diagdown & & | \\ X & - & D & - & X & - & E \end{array} \right] = \begin{array}{ccccc} A & & X & - & B & & C \\ | & \diagup & | & & \diagdown & & | \\ \boxed{X} & - & \boxed{D} & - & \boxed{X} & - & E \end{array} = \begin{array}{c} A \quad \bigcap \\ \quad \quad \quad \diagup \quad \diagdown \\ \quad \quad \quad M_3 \\ \quad \quad \quad \diagdown \quad \diagup \\ D \quad \quad \quad \diagdown \quad \diagup \\ \quad \quad \quad B \quad C \\ \quad \quad \quad | \\ \quad \quad \quad E \end{array}$$

其中 M_3 是期望¹ $\left[\begin{smallmatrix} | & | & | \\ \diagdown & \diagup & \diagdown \end{smallmatrix} \right]$ ，它是一个 9 阶张量，并且不依赖常量 A 、 B 、 C 和 D 。在实践中，你会想给边命名，以便追踪哪些对象与哪些对象相乘。

¹FIXME: 这与刚刚引入的记号不同。

我们甚至可以利用期望的线性性，把期望推进到张量的无穷和内部；如下矩生成函数把所有 M_k 张量联系起来：

$$\begin{aligned} E(e^{\langle x \ominus m, t \rangle}) &= \sum_{k=0}^{\infty} \frac{1}{k!} E[\langle x \ominus m, t \rangle^k] = E\left[\sum_{k=0}^{\infty} \frac{1}{k!} \langle (x \ominus m)^{\otimes k}, t^{\otimes k} \rangle\right] = \sum_{k=0}^{\infty} \frac{1}{k!} \langle M_k, t^{\otimes k} \rangle \\ &= 1 + \underset{m}{\overset{t}{|}} + \frac{1}{2} \underset{t}{\overset{t}{|}} M \underset{t}{\overset{t}{|}} + \frac{1}{6} \underset{t}{\overset{t}{|}} M_3 \underset{t}{\overset{t}{|}} + \frac{1}{24} \underset{t}{\overset{t}{|}} M_4 \underset{t}{\overset{t}{|}} + \dots \end{aligned}$$

5.1.2 线性形式

《Matrix Cookbook》给出了下面这个简单的期望：

$$E[AXB + C] = AE[X]B + C \quad \begin{bmatrix} -A-X-B- \\ + -C- \end{bmatrix} = \begin{matrix} -A-[X]-B- \\ + -C- \end{matrix} \quad (312)$$

5.1.3 二次型

我们常常更愿意用简单的中心矩来写期望；这可以通过提出均值来做到：

$$\begin{bmatrix} x- \\ x- \end{bmatrix} = \begin{bmatrix} (x \ominus m)- \\ (x \ominus m)- \end{bmatrix} + \begin{matrix} m- \\ m- \end{matrix} = M \underset{m-}{\overset{m-}{\lhd}} + \begin{matrix} m- \\ m- \end{matrix}$$

这样就很容易处理《Matrix Cookbook》中的二次型：

$$\begin{aligned} \text{Var}[Ax] &= A \text{Var}[x] A^T \quad \begin{bmatrix} A-x \ominus [A-x] \\ A-x \ominus [A-x] \end{bmatrix} = \begin{bmatrix} A-(x \ominus m) \\ A-(x \ominus m) \end{bmatrix} \\ &= \begin{matrix} A- \\ A- \end{matrix} \begin{bmatrix} (x \ominus m) \\ (x \ominus m) \end{bmatrix} \\ &= -A-M_2-A- \end{aligned} \quad (313)$$

$$\begin{aligned} E[x^T A x] &= \text{Tr}(AM) + m^T A m \quad [x-A-x] = \left(\begin{bmatrix} (x \ominus m)- \\ (x \ominus m)- \end{bmatrix} + \begin{matrix} m- \\ m- \end{matrix} \right) \underset{A}{\rhd} \\ &= \begin{bmatrix} (x \ominus m) \\ (x \ominus m) \end{bmatrix} \underset{A}{\rhd} + \begin{matrix} m- \\ m- \end{matrix} \underset{A}{\rhd} \end{aligned} \quad (318)$$

$$= \overbrace{M-A} + m-A-m$$

$$E[(A\mathbf{x} + a)(B\mathbf{x} + b)^T] = AMB^T + (Am + a)(Bm + b)^T \quad (320)$$

$$E[\mathbf{x}\mathbf{x}^T] = M + mm^T \quad (321)$$

$$E[\mathbf{x}a^T\mathbf{x}] = (M + mm^T)a \quad (322)$$

$$E[\mathbf{x}^T a\mathbf{x}^T] = a^T(M + mm^T) \quad (323)$$

$$E[(A\mathbf{x})(A\mathbf{x})^T] = A(M + mm^T)A^T \quad (324)$$

$$E[(\mathbf{x} + a)(\mathbf{x} + a)^T] = M + (m + a)(m + a)^T \quad (325)$$

$$E[(A\mathbf{x} + a)^T(B\mathbf{x} + b)] = \text{Tr}(AMB^T) + (Am + a)^T(Bm + b) \quad (326)$$

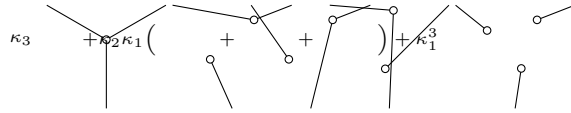
$$E[\mathbf{x}^T\mathbf{x}] = \text{Tr}(M) + m^T m \quad (327)$$

$$E[\mathbf{x}^T A\mathbf{x}] = \text{Tr}(AM) + m^T Am \quad (328)$$

$$E[(A\mathbf{x})^T(A\mathbf{x})] = \text{Tr}(AMA^T) + (Am)^T(Am) \quad (329)$$

$$E[(\mathbf{x} + a)^T(\mathbf{x} + a)] = \text{Tr}(M) + (m + a)^T(m + a) \quad (330)$$

5.1.4 三次型



当 x 是均值向量为 m 的随机向量时，把原始三阶矩展开为中心矩会很方便：

$$\begin{aligned} \begin{bmatrix} x- \\ x- \\ x- \end{bmatrix} &= \begin{bmatrix} (x \ominus m)- \\ (x \ominus m)- \\ (x \ominus m)- \end{bmatrix} + 3 \begin{bmatrix} (x \ominus m)- \\ m- \\ m- \end{bmatrix} + 3 \begin{bmatrix} m- \\ (x \ominus m)- \\ (x \ominus m)- \end{bmatrix} + \begin{bmatrix} m- \\ m- \\ m- \end{bmatrix} \\ &= \overset{|}{\underset{\diagup \diagdown}{M_3}} + 3 \begin{matrix} m- \\ -M- \end{matrix} + \begin{matrix} m- \\ m- \end{matrix} \end{aligned}$$

TODO: m, M 项中的边需要做对称化。

但这仍然有些杂乱。另见下面关于累积量的内容。

假设 $\mathbf{x}$ 是一个坐标相互独立的随机向量，均值为 m ，协方差为 M ，中心矩为 $v_3 = \mathbb{E}[(\mathbf{x} - m)^3]$ 。那么（见 [7]）

$$\begin{aligned}
 & \mathbb{E}[(A\mathbf{x} + a)(B\mathbf{x} + b)^T(C\mathbf{x} + c)] \\
 &= A \operatorname{diag}(B^T C) v_3 \\
 &+ (Am + a) \operatorname{Tr}(BMC^T) \\
 &+ AMC^T(Bm + b) \\
 &+ AMB^T(Cm + c) \\
 &+ (Am + a)(Bm + b)^T(Cm + c) \\
 &\mathbb{E}[\mathbf{xx}^T \mathbf{x}] \\
 &= v_3 \\
 &+ 2Mm \\
 &+ (\operatorname{Tr}(M) + m^T m)m
 \end{aligned}$$

5.2 累积量

给定随机向量 $x \in \mathbb{R}^d$ ，它的 n 阶累积量张量 $K_n \in \mathbb{R}^{d, \dots, d}$ 定义为

$$\log \mathbb{E}(e^{\langle t, x \rangle}) = \sum_{n=1}^{\infty} \frac{1}{n!} \langle K_n, t^{\otimes n} \rangle = \frac{t}{K_1} + \frac{1}{2} \frac{t}{K_2} + \frac{1}{6} \frac{t}{t' \backslash t} + \dots$$

前几个累积量类似于中心矩：

$$K_1 = m \quad \text{和} \quad K_2 = M \quad \text{和} \quad K_3 = M_3 \quad \text{和} \quad K_4 = M_4 - \frac{-M_2-}{-M_2-} - \frac{\backslash M_2}{\backslash M_2} \frac{\backslash}{\backslash} \frac{M_2'}{\backslash M_2'} \frac{\backslash}{\backslash} \frac{M_2'}{\backslash M_2'}.$$

它们有一个很好的性质（从定义很容易看出）：对于独立随机变量，累积量是可加的： $K_n(x+y) = K_n(x) + K_n(y)$ 。这推广了一个标准性质：独立随机变量之和的方差等于方差之和。

我们可以用累积量来写出 $x^{\otimes n}$ 的期望：

$$\left[\begin{array}{c} x \quad x \\ x \\ | \end{array} \right] = \frac{\backslash K_3}{|} + \frac{\backslash K_2}{K_1} + \frac{\backslash K_2}{K_1} + \frac{\backslash K_1}{K_2} + \frac{\backslash K_1}{K_1} \frac{K_1'}{K_1'}.$$

一般而言，求和遍历集合 $\{1, \dots, n\}$ 的所有划分。

如果 x 的各个分量相互独立, 累积量张量 $K_1, K_2, \dots$ 的非对角项为零。这意味着可以按如下方式写出: 假设每个 x_i 的累积量为 $\kappa_1, \kappa_2, \kappa_3, \kappa_4 \in \mathbb{R}$, 则

$$\begin{aligned} \begin{bmatrix} x & x \\ x & x \end{bmatrix} &= \kappa_4 \text{ (four-point star)} \\ &+ \kappa_3 \kappa_1 \left(\text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} \right) \\ &+ \kappa_2^2 \left(\text{two-point star} + \text{two-point star} + \text{two-point star} \right) \\ &+ \kappa_2 \kappa_1^2 \left(\text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} \right) \\ &+ \kappa_1^4 \text{ (four-point star)} \end{aligned}$$

特别注意, 如果均值 κ_1 为零, 就只剩下四项。当 $n = 5$ 时, 总共有 52 个划分; 但若 $\kappa_1 = 0$, 只有 11 项保留下来:

$$\kappa_5 \text{ (five-point star)} + \kappa_3 \kappa_2 \left(\text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} \right)$$

如果 x 是高斯变量, 所有 3 阶及更高阶累积量都为零。当 $n = 6$ 时, 总共有 203 个划分, 但 $E[x^{\otimes 6}]$ 只剩 15 项²:

$$\begin{aligned} &\kappa_2^3 \left(\text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} \right. \\ &\quad \left. + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} + \text{two-point star} \right). \end{aligned}$$

5.2.1 四次型

我们可以用它来计算 $\text{Var}[x^T A x]$ 。假设 A 是对称的:

$$\begin{aligned} \text{Var}[x^T A x] &= \begin{bmatrix} x & A & x \\ x & A & x \end{bmatrix} - \begin{bmatrix} x & A & x \\ x & A & x \end{bmatrix} \\ &= \kappa_4 A \text{ (four-point star)} + 4\kappa_3 \kappa_1 A \text{ (two-point star)} - A \text{ (two-point star)} \\ &\quad + 2\kappa_2^2 \text{ (two-point star)} + \kappa_2^2 \text{ (two-point star)} \\ &\quad + 4\kappa_2 \kappa_1^2 \text{ (two-point star)} - A \text{ (two-point star)} + 2\kappa_2 \kappa_1^2 \text{ (two-point star)} - A \text{ (two-point star)} \\ &\quad + \kappa_1^4 \text{ (two-point star)} - A \text{ (two-point star)} - A \text{ (two-point star)} \\ &\quad - \left(\kappa_2 \text{ (two-point star)} + \kappa_1^2 \text{ (two-point star)} \right)^2 \\ &= 2\kappa_2^2 \text{ (two-point star)} + 4\kappa_2 \kappa_1^2 \text{ (two-point star)} - A \text{ (two-point star)} \\ &\quad + 4\kappa_3 \kappa_1 A \text{ (two-point star)} - A \text{ (two-point star)} + \kappa_4 A \text{ (four-point star)} \end{aligned}$$

²这就是为什么对 $g \sim N(0, 1)$ 有 $E[g^6] = 15$ 。

《Matrix Cookbook》把它列为

$$\text{Var}[x^T A x] = 2\mu_2^2 \text{Tr}(A^2) + 4\mu_2 c^T A^2 c + 4\mu_3 c^T A a + (\mu_4 - 3\mu_2^2) a^T a \quad (319)$$

其中 $c = \mu_1$ 是 x 的均值, $a = \text{diag}(A)$ 是 A 的对角线, 而 $\mu_4 = \kappa_4 + 3\kappa_2^2$ 是四阶中心矩。

5.3 加权标量变量

令 $y = w^T x$, 并令 $m = E[y]$, 则

$$E[y] = m = w^T \mu \quad (321)$$

$$E[(y \ominus m)^2] = w - M_2 - w \quad (322)$$

$$E[(y \ominus m)^3] = w - \overset{w}{\underset{w}{M_3}} - w \quad (323)$$

$$E[(y \ominus m)^4] = w - \overset{w}{\underset{w}{M_4}} - w \quad (324)$$

对于 $x \sim N(0, 1)$, 有不等式:

$$E[y^n]^{1/n} \leq \sqrt{2/\pi} \|w\|_2.$$

5.4 高斯矩

对于高斯向量 $x \sim \mathcal{N}(m, M)$: - 所有奇数阶中心矩都为零: $M_3 = 0$, 等等。 - 偶数阶矩可以通过 Isserlis 定理计算。

例如:

$$\mathbb{E}[(x \ominus m)^{\otimes 4}] = M \otimes M + M \otimes M + \dots$$

(对指标的不同配对求和)。

5.4.1 高斯分部积分

如果 X 是一个由高斯条目组成、均值为零且带有某个协方差的张量, 那么对任意可微函数 f , Stein 引理给出下面这个非常一般的方程:

结合第四章中的张量链式法则，这可以成为求解许多困难期望的强大方法。

$$E(xx^T) = \Sigma + mm^T \quad (377)$$

$$E[x^T Ax] = \text{Tr}(A\Sigma) + m^T Am \quad (378)$$

$$\begin{aligned} \text{Var}(x^T Ax) &= \text{Tr}[A\Sigma(A + A^T)\Sigma] + \dots \\ &\quad + m^T(A + A^T)\Sigma(A + A^T)m \end{aligned} \quad (379)$$

$$E[(x - m')^T A(x - m')] = (m - m')^T A(m - m') + \text{Tr}(A\Sigma) \quad (380)$$

如果 $\Sigma = \sigma^2 I$ 且 A 对称，则

$$\text{Var}(x^T Ax) = 2\sigma^4 \text{Tr}(A^2) + 4\sigma^2 m^T A^2 m \quad (381)$$

假设 $x \sim \mathcal{N}(0, \sigma^2 I)$ ，且 A 和 B 对称，则

$$\text{Cov}(x^T Ax, x^T Bx) = 2\sigma^4 \text{Tr}(AB) \quad (382)$$

5.4.2 三次型

假设 x 是一个坐标相互独立的随机向量，均值为 m ，协方差为 M ：

$$\begin{aligned} E[xb^T xx^T] &= mb^T(M + mm^T) + (M + mm^T)bm^T \\ &\quad + b^T m(M - mm^T) \end{aligned} \quad (383)$$

5.4.3 四次型的均值

$$\begin{aligned} E[xx^T xx^T] &= 2(\Sigma + mm^T)^2 + m^T m(\Sigma - mm^T) \\ &\quad + \text{Tr}(\Sigma)(\Sigma + mm^T) \end{aligned}$$

$$\begin{aligned} E[xx^T Axx^T] &= (\Sigma + mm^T)(A + A^T)(\Sigma + mm^T) \\ &\quad + m^T Am(\Sigma - mm^T) + \text{Tr}[A\Sigma](\Sigma + mm^T) \end{aligned}$$

$$E[x^T xx^T x] = 2\text{Tr}(\Sigma^2) + 4m^T \Sigma m + (\text{Tr}(\Sigma) + m^T m)^2$$

$$\begin{aligned} E[x^T A x x^T B x] &= \text{Tr}[A \Sigma (B + B^T) \Sigma] + m^T (A + A^T) \Sigma (B + B^T) m \\ &\quad + (\text{Tr}(A \Sigma) + m^T A m)(\text{Tr}(B \Sigma) + m^T B m) \end{aligned}$$

$$\begin{aligned} E[a^T x b^T x c^T x d^T x] &= (a^T (\Sigma + m m^T) b)(c^T (\Sigma + m m^T) d) \\ &\quad + (a^T (\Sigma + m m^T) c)(b^T (\Sigma + m m^T) d) \\ &\quad + (a^T (\Sigma + m m^T) d)(b^T (\Sigma + m m^T) c) - 2a^T m b^T m c^T m d^T m \end{aligned}$$

$$\begin{aligned} E[(Ax + a)(Bx + b)^T(Cx + c)(Dx + d)^T] &= [A \Sigma B^T + (Am + a)(Bm + b)^T][C \Sigma D^T + (Cm + c)(Dm + d)^T] \\ &\quad + [A \Sigma C^T + (Am + a)(Cm + c)^T][B \Sigma D^T + (Bm + b)(Dm + d)^T] \\ &\quad + (Bm + b)^T(Cm + c)[A \Sigma D^T - (Am + a)(Dm + d)^T] \\ &\quad + \text{Tr}(B \Sigma C^T)[A \Sigma D^T + (Am + a)(Dm + d)^T] \end{aligned}$$

$$\begin{aligned} E[(Ax + a)^T(Bx + b)(Cx + c)^T(Dx + d)] &= \text{Tr}[A \Sigma (C^T D + D^T C) \Sigma B^T] \\ &\quad + [(Am + a)^T B + (Bm + b)^T A] \Sigma [C^T (Dm + d) + D^T (Cm + c)] \\ &\quad + [\text{Tr}(A \Sigma B^T) + (Am + a)^T (Bm + b)] [\text{Tr}(C \Sigma D^T) + (Cm + c)^T (Dm + d)] \end{aligned}$$

见 [7]。

5.4.4 高斯混合

对于高斯混合：

$$x \sim \sum_k \pi_k \mathcal{N}(m_k, M_k),$$

矩是加权和：

$$E[x] = \sum_k \pi_k m_k, \quad \text{Var}[x] = \sum_k \pi_k (M_k + m_k m_k^T) - \left(\sum_k \pi_k m_k \right) \left(\sum_k \pi_k m_k \right)^T.$$

高阶矩也类似地通过线性组合。

$$E[x] = \sum_k \rho_k m_k \quad (384)$$

$$\text{Cov}(x) = \sum_k \sum_{k'} \rho_k \rho_{k'} (\Sigma_k + m_k m_k^T - m_k m_{k'}^T) \quad (385)$$

5.4.5 导数

矩关于 m 或 M 的导数可以通过在积分号下求导得到；在高斯情形下，还可以使用 Stein 引理。

5.5 向量与张量的自由概率

自由概率理论提供了经典概率的非交换类比。我们将从最简单的情形开始，即条目为标量的向量；然后推进到矩阵，最后到更高阶张量。

5.5.1 向量的自由累积量

对于条目为标量的随机向量 $x \in \mathbb{R}^n$ ，自由累积量的定义类似于经典累积量，但只使用非交叉划分。

向量的 R-变换

R-变换（自由累积量生成函数）为：

$$R_x(z) = \sum_{n=1}^{\infty} \kappa_n^{\text{free}} z^{n-1}$$

这是经典累积量生成函数 $\log E[e^{tz}] = \sum_{n=1}^{\infty} \frac{\kappa_n}{n!} t^n$ 的自由概率对应物。

矩-累积量关系

如果每个 x_i 的自由累积量为 $\kappa_1^{\text{free}}, \kappa_2^{\text{free}}, \kappa_3^{\text{free}}, \kappa_4^{\text{free}} \in \mathbb{R}$, 则:

$$\begin{bmatrix} x & x \\ x & x \end{bmatrix}_{\text{free}} = \begin{aligned} & + \kappa_3^{\text{free}} \kappa_1^{\text{free}} \left(\text{diagram 1} \right) \\ & + (\kappa_2^{\text{free}})^2 \left(\text{diagram 2} \right) \\ & + \kappa_2^{\text{free}} (\kappa_1^{\text{free}})^2 \left(\text{diagram 3} \right) \\ & + (\kappa_1^{\text{free}})^4 \left(\text{diagram 4} \right) \end{aligned}$$

红色 = 交叉划分 (在自由概率中排除)

与经典情形相比: 15 个划分 $\rightarrow$ 9 个非交叉划分!

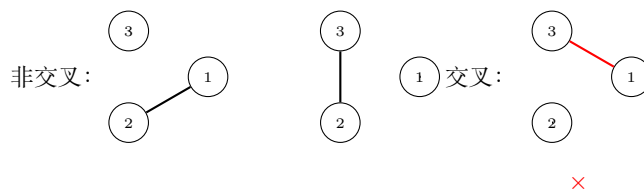
5.5.2 矩阵的自由累积量

对于随机矩阵, 我们必须考虑乘积的迹。与标量不同, 矩阵乘积不满足交换律, 这增加了复杂性。

$$\text{经典累积量: } E[X^3] = \kappa_3 + 3\kappa_2\kappa_1 + \kappa_1^3 \quad (5 \text{ 个划分})$$

$$\text{自由累积量: } E[\text{Tr}(X^3)] = \kappa_3^{\text{free}} + \kappa_1^{\text{free}} \kappa_2^{\text{free}} \quad (2 \text{ 个 NC 划分})$$

划分 (1, 3), (2) 是交叉的, 因此在自由概率中被排除:



关键洞见: 对矩阵来说, 元素之间只有一条边需要匹配。对更高阶张量来说, 则有多条边 (颜色) 需要匹配。

5.5.3 张量自由累积量

对于 D 阶张量, 由于有彩色非交叉划分, 自由累积量理论变得更加丰富。

张量的彩色非交叉划分

对于 D 阶张量, 每个元素有 D 条“腿”或指标。一个有效配对必须:

- 当元素排在圆周上时是非交叉的
- 在被配对元素之间匹配全部 D 种颜色 (每条腿一种)

这会大幅减少有效划分的数量。对于 3 阶张量:

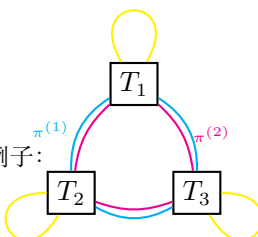
矩阵: $\boxed{X} \overset{1 \text{ 条边}}{\text{---}} \boxed{X}$ 3 阶张量: $\boxed{T} \overset{3 \text{ 条边}}{\text{---}} \boxed{T}$

张量的不变量迹

不同于矩阵, 高阶张量没有唯一的标准迹。相反, 每一种把张量缩并成标量的方式都会定义一个不同的不变量:

矩阵 ($D = 2$)	张量 ($D > 2$)
唯一不变量 $\text{Tr}(X_1 \cdots X_n)$	不变量族 $\text{Tr}_\pi(T_1, \dots, T_n)$ 由彩色置换 $\pi \in \mathfrak{S}_{Dn}$ 索引

Tr_π 的定义: 给定张量 $T_1, \dots, T_n$, 每个张量都有 D 条腿 (颜色从 1 到 D), 彩色置换 $\pi = (\pi^{(1)}, \dots, \pi^{(D)}) \in \mathfrak{S}_n^D$ 指定哪些腿相连:

$\text{Tr}_\pi(T_1, \dots, T_n) =$ 例子: 

对于矩阵 ($D = 2$), 只有循环置换会给出非零迹, 从而恢复通常的 $\text{Tr}(X_1 X_2 \cdots X_n)$ 。

归一化: 为了在 $N \rightarrow \infty$ 时得到有限极限, 我们用 $N^{-(D-1)n}$ 归一化:

$$\frac{1}{N^{(D-1)n}} \text{Tr}_\pi(T_1, \dots, T_n) \xrightarrow{N \rightarrow \infty} \text{finite}$$

矩-累积量公式

张量的矩-累积量关系涉及所有不变量迹：

$$\left(\mathbb{E}[\text{Tr}_\pi(T_{i_1}, \dots, T_{i_n})] \right)_{\pi \in \text{NC}_D(n)} = \sum_{\sigma \in \text{NC}_D(n)} \kappa_\sigma^{(t)} \cdot (\text{pattern matching})$$

这些矩形成一个由彩色非交叉置换索引的向量，而张量自由性作用在整个族上。

$n = 2$ 的例子：二阶矩分解为

$$\mathbb{E}[\text{Tr}(T_1 T_2)] = \kappa_2^{(t)} \begin{array}{c} \textcircled{1} \\ \textcircled{2} \end{array} \delta_{i_1, i_2} + (\kappa_1^{(t)})^2 \begin{array}{c} \textcircled{1} \\ \textcircled{2} \end{array}$$

对于高斯张量， $\kappa_1^{(t)} = 0$ 且当 $n > 2$ 时 $\kappa_n^{(t)} = 0$ ，因此：

$$\mathbb{E}[\text{Tr}(T_1 T_2)] = \kappa_2^{(t)} \delta_{i_1, i_2} = \delta_{i_1, i_2}$$

5.5.4 张量 Wick 定理

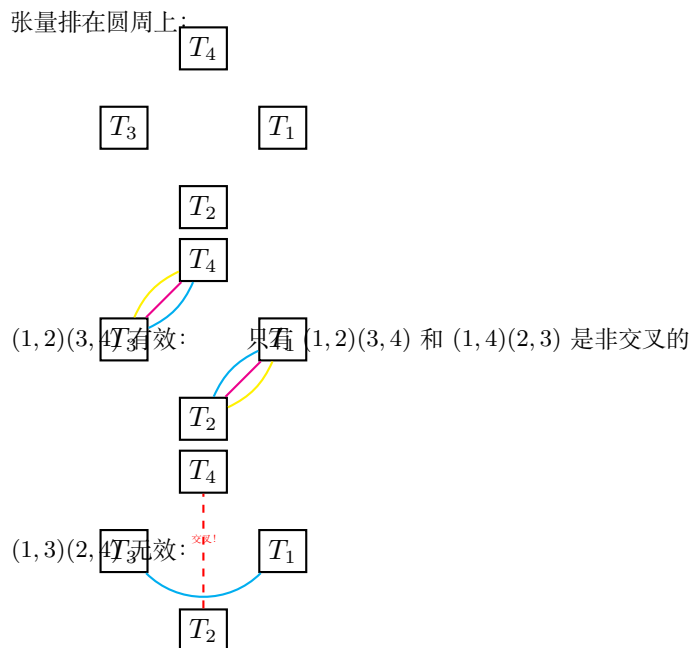
对于高斯张量，一个类似 Wick 定理的基本结果成立：

张量 Wick 分解：设 $(T_i)_{i \in I}$ 是大小为 N 的独立同分布标准高斯 D 腿张量。当 $N \rightarrow \infty$ 时：

$$\mathbb{E}[T_{i_1} \cdots T_{i_{2k}}] = \sum_{\pi \in \text{NC}_D(2k)} \prod_{(j, j') \in \pi} \delta_{i_j, i_{j'}} + O(N^{-1})$$

这说明高斯张量渐近张量自由，并且二阶以上的所有累积量都消失。

对于四个 3 阶张量, 有效的彩色非交叉配对受到严格限制:



5.5.5 张量自由性下的可加性

如果两个张量代数 $\mathcal{A}_1$ 和 $\mathcal{A}_2$ 是张量自由的, 那么对 $A \in \mathcal{A}_1$ 和 $B \in \mathcal{A}_2$ 有:

$$\kappa_n^{(t)}(A + B) = \kappa_n^{(t)}(A) + \kappa_n^{(t)}(B)$$

这推广了独立变量的经典累积量可加性。

图示地看, 对于二阶累积量 (协方差):

$$\kappa_2^{(t)}\left(\begin{array}{c} \boxed{A} \\ \boxed{B} \end{array}\right) = \kappa_2^{(t)}[\boxed{A}] + \kappa_2^{(t)}[\boxed{B}]$$

混合累积量消失

对于矩阵, 如果 A 和 B 自由且其中一个迹为零, 那么交替乘积满足 $\kappa_n^{\text{free}}(AB \cdots) = 0$ 。对于张量, 类似性质取决于缩并模式。

定义: 如果 $\kappa_1^{(t)}(T) = 0$, 则称张量 T 具有零一阶累积量; 这意味着对所有彩色置换 $\pi \in \mathfrak{S}_D$, 都有 $E[\text{Tr}_\pi(T)] = 0$ 。

定义: 交替缩并模式是连接序列 $A_1, B_1, A_2, B_2, \dots$ 中张量的一种方式, 其中:

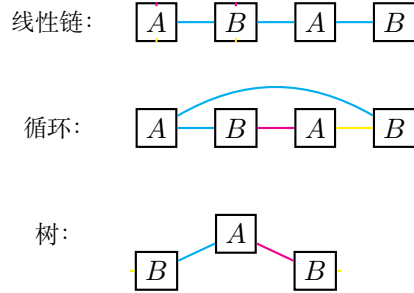
- 张量以交替顺序出现 ($A, B, A, B, \dots$)

- 每个张量的腿都与相邻张量缩并（没有张量孤立）
- 缩并尊重张量结构

定理：如果 A 和 B 张量自由，且 $\kappa_1^{(t)}(A) = 0$ ，那么任意交替缩并的累积量都为零：

$$\kappa_n^{(t)}(A \otimes_C B \otimes_C A \otimes_C \cdots) = 0 \quad \text{for } n \geq 2$$

3 阶张量的交替缩并示例：



如果 $\kappa_1^{(t)}(A) = 0$ ，这些缩并都会给出 $\kappa_n^{(t)} = 0$ 。

这是因为任意非交叉划分都必须包含一个含有 A 的单元素块，但 $\kappa_1^{(t)}(A) = 0$ 。

与矩阵的关键差异在于：对于张量，我们必须指定缩并模式，因为即使是同一批张量，不同缩并也可能产生不同结果。

5.5.6 张量 Wishart 矩阵的谱

考虑一个 3 阶张量 $T \in \mathbb{C}^{N \times N \times N}$ ，其条目为独立同分布的高斯变量 $\mathcal{N}_{\mathbb{C}}(0, N^{-3})$ 。Wishart 型矩阵 $W = TT^\dagger$ （缩并一个指标）具有如下极限谱分布：

$$\boxed{T} \quad \boxed{T^\dagger} \xrightarrow{\text{MP}(1) \boxtimes \text{MP}(1)} \text{MP}(1/2)$$

其中 $\boxtimes$ 表示自由乘性卷积， $\text{MP}(c)$ 是 Marchenko-Pastur 分布。

5.5.7 张量网络与自由概率

自由概率为分析张量网络态提供了强大工具，而张量网络态自然出现在量子多体物理和机器学习中。

图 amalgamation (合并)

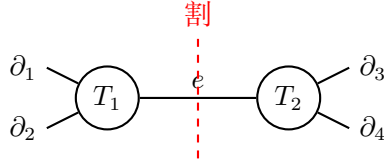
对于图 $G = (V, E)$ 上的张量网络, 如果每个顶点放置随机高斯张量, 则边界密度矩阵会按照图结构分解:

图 amalgamation (合并): 令 ρ_b 为随机张量网络的边界密度矩阵。则:

$$\rho_b = \bigstar_{\gamma \in \mathcal{C}_{\min}} \rho_\gamma$$

其中 $\bigstar_{\mathcal{B}}$ 表示在边界代数上的带 amalgamation (合并) 的自由乘性卷积, $\mathcal{C}_{\min}$ 是 G 的最小割。

例如, 一个简单的双顶点网络:

**纠缠与最大流**

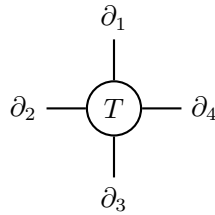
张量网络态的 Rényi 熵与网络图上的最大流问题相联系:

$$S_q(\rho_b) = \frac{1}{1-q} \log \left[\max_f \prod_{e \in E} w_e^{f(e)} \right] + o(1)$$

其中 f 遍历整数流, w_e 是边权。对于最大纠缠连接 ($w_e = 1$), 这会恢复全息理论中的 Ryu-Takayanagi 公式。

5.5.8 高阶张量示例**星形网络**

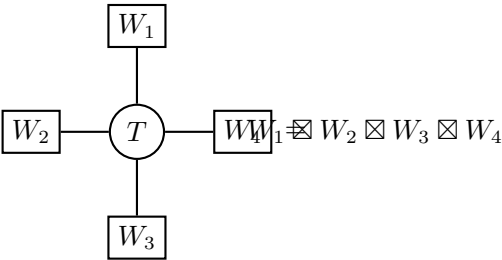
考虑一个星形图, 其中有一个度数为 $D = 4$ 的中心顶点:



边界密度矩阵是 $\rho_b = W_1 W_2 W_3 W_4$, 其中包含独立的 Wishart 因子。由张量自由性可得:

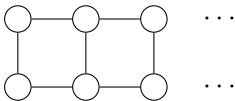
$$\mu_\rho = \text{MP}(1)^{\boxtimes 4}, \quad R_\rho(w) = \frac{4w}{1-w}$$

每个格点的 von Neumann 熵等于 $\log 4 - 3/4$ 。
自由积分解为：

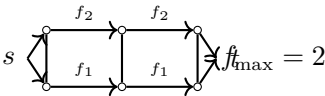


梯形网络

对于带周期竖直边界的 $2 \times L$ 梯形网络：



纠缠熵按 $S_2 = \lceil L/2 \rceil \log N + O(1)$ 缩放，表现出面积律缩放。
穿过网络的最大流为：



5.5.9 D 元 Catalan 数

$\text{NC}_D(n)$ 的彩色 Möbius 函数涉及 D 元 Catalan 数：

$$\mu_{\text{col}}(0_n, 1_n) = (-1)^{n-1} C_{n-1}^{(D)}$$

其中 $C_n^{(D)} = \frac{1}{(D-1)n+1} \binom{Dn}{n}$ 是 Fuss-Catalan 数。
 D 元 Catalan 数的增长：

n	$C_n^{(2)}$	$C_n^{(3)}$	$C_n^{(4)}$
0	1	1	1
1	2	3	5
2	5	12	35
3	14	55	285

当 $D = 2$ (矩阵) 时，我们恢复标准 Catalan 数。对于 $D = 3$ (3 阶张量)：

$$C_0^{(3)} = 1, \quad C_1^{(3)} = 1, \quad C_2^{(3)} = 3, \quad C_3^{(3)} = 12, \quad C_4^{(3)} = 55, \dots$$

5.5.10 Melon 型图

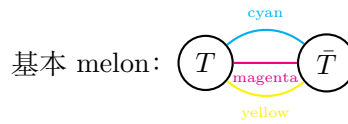
Melonic diagrams (melon 型图) 是张量模型的 $1/N$ 展开中的主导阶 Feynman 图。它们得名于类似 melon 的形状，并具有特殊的拓扑结构。

定义与结构

对于 D 阶张量，melon 型图是可以画在 2-球面上的图，并满足：

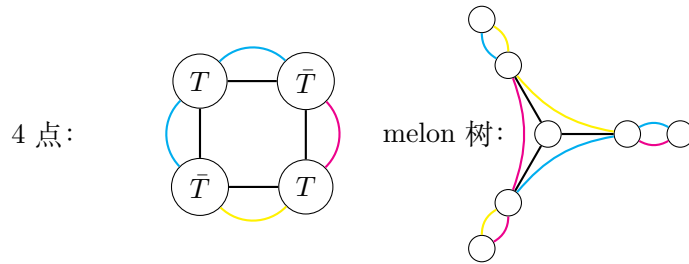
- 每个面由 D 种颜色之一着色
- 图具有由 “melons” 组成的树状结构
- 任意顶点周围没有颜色重复出现

3 阶张量的基本 melon (2 点函数)：



高阶 melon 型图

4 点 melon 型图 (N^{-1} 阶)：



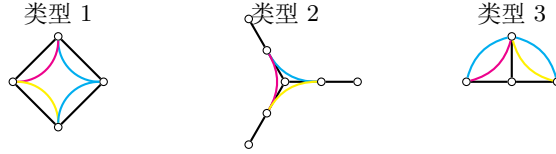
计数 melon 型图

melon 型图的数量增长远慢于一般图：

Melonic 主导性：在 $1/N$ 展开的 n 阶：

- 一般图： $\sim (D!)^n n!$
- Melon 型图： $\sim D^n \cdot C_{n-1}$ (Catalan 数)

对于 3 阶张量的 3 阶项:



Melonic 普适性

在大 N 极限下, 张量模型由 melon 型图主导, 这会导致:

- 谱维数 $d_s = 2$ (类似分支聚合物)
- 当 $D \geq 3$ 时, 临界指数与 D 无关
- 与物理中的 Sachdev-Ye-Kitaev (SYK) 模型相联系

这种普适性正是 melon 型图在理解张量模型及其连续极限时处于核心地位的原因。

5.6 练习

练习 41.

$$\begin{aligned} \mathbb{E}[(\mathbf{A}\mathbf{x} + a)(\mathbf{A}\mathbf{x} + a)^T(\mathbf{A}\mathbf{x} + a)] &= \text{A} \text{diag}(\mathbf{A}^T \mathbf{A}) v_3 \\ &+ [2\mathbf{A} \mathbf{M} \mathbf{A}^T + (\mathbf{A}\mathbf{x} + a)(\mathbf{A}\mathbf{x} + a)^T](\mathbf{A} \mathbf{m} + a) \\ &+ \text{Tr}(\mathbf{A} \mathbf{M} \mathbf{A}^T)(\mathbf{A} \mathbf{m} + a) \end{aligned}$$

$$\begin{aligned} \mathbb{E}[(\mathbf{A}\mathbf{x} + a)b^T(\mathbf{C}\mathbf{x} + c)(\mathbf{D}\mathbf{x} + d)^T] &= (\mathbf{A}\mathbf{x} + a)b^T(\mathbf{C} \mathbf{M} \mathbf{D}^T + (\mathbf{C} \mathbf{m} + c)(\mathbf{D} \mathbf{m} + d)^T) \\ &+ (\mathbf{A} \mathbf{M} \mathbf{C}^T + (\mathbf{A} \mathbf{m} + a)(\mathbf{C} \mathbf{m} + c)^T)b(\mathbf{D} \mathbf{m} + d)^T \\ &+ b^T(\mathbf{C} \mathbf{m} + c)(\mathbf{A} \mathbf{M} \mathbf{D}^T - (\mathbf{A} \mathbf{m} + a)(\mathbf{D} \mathbf{m} + d)^T) \end{aligned}$$

练习 42. 在 Matrix Reference Manual 中寻找更多恒等式, 并尝试证明它们。也请尝试用 Tensorgrad 验证你的推导。

练习 43. 对于带彩色腿 (cyan、magenta、yellow) 的 3 阶张量 T_1, T_2, T_3 , 画出 $\{1, 2, 3\}$ 的所有有效 3 色非交叉划分。与集合划分总数相比, 它们有多少个?

练习 44. 考虑一个 4 阶高斯张量 $T \in \mathbb{R}^{N \times N \times N \times N}$ 。计算 $\mathbb{E}[\text{Tr}_{12}(T)\text{Tr}_{34}(T)]$, 其中 Tr_{ij} 表示对指标 i 和 j 的缩并。使用张量 Wick 定理, 并画出相应的彩色图。

练习 45. 对于深度为 d 的二叉树上的张量网络，证明边界纠缠熵按 $S \sim d \log N$ 缩放。使用图 amalgamation 定理，并找出最小割。

练习 46. 验证对于 D 阶张量， n 阶 melon 型图的数量按 $D^n n!$ 增长。画出 $D = 3, n = 3$ 时的所有 melon 型图。

练习 47. 证明 D 元 Catalan 数满足递推式：

$$C_n^{(D)} = \sum_{k=0}^{n-1} C_k^{(D)} C_{n-1-k}^{(D-1)}$$

这对彩色非交叉划分的结构意味着什么？

练习 48. 对于两个张量自由的 3 阶张量 A 和 B ，计算第三张量累积量 $\kappa_3^{(t)}(A + B)$ 。使用彩色划分图验证可加性性质。

第六章 行列式与逆矩阵

6.1 行列式

用张量记号把行列式写成如下形式很方便：

$$\det(A) = \frac{1}{n!} \overline{\overline{A \cdots A}}$$

其中 $\overline{\overline{i_1 \ i_2 \ \cdots \ i_n}} = \varepsilon_{i_1, \dots, i_n}$ 是如下定义的 n 阶 Levi-Civita 张量：

$$\varepsilon_{i_1, \dots, i_n} = \begin{cases} \text{sign}(\sigma) & \sigma = (i_1, \dots, i_n) \text{ 是一个置换} \\ 0 & \text{其他情况。} \end{cases}$$

为了看出这个定义是合理的，先考虑

$$\det(I) = \frac{1}{n!} \overline{\overline{\begin{bmatrix} \square & \cdots & \square \end{bmatrix}}} = \frac{1}{n!} \sum_{i_1, \dots, i_n, j_1, \dots, j_n} \varepsilon_{i_1, \dots, i_n} \varepsilon_{j_1, \dots, j_n} [i = j] = \frac{1}{n!} \sum_{i_1, \dots, i_n} \varepsilon_{i_1, \dots, i_n}^2 = 1.$$

一般地，由行列式的置换定义可得：

$$\begin{aligned} \overline{\overline{A \cdots A}} &= \sum_{i_1, \dots, i_n, j_1, \dots, j_n} \varepsilon_{i_1, \dots, i_n} \varepsilon_{j_1, \dots, j_n} A_{i_1, j_1} \cdots A_{i_n, j_n} \\ &= \sum_{\sigma, \tau} \text{sign}(\sigma) \text{sign}(\tau) A_{\sigma_1, \tau_1} \cdots A_{\sigma_n, \tau_n} \\ &= \sum_{\sigma} \text{sign}(\sigma) \sum_{\tau} \text{sign}(\tau) A_{\sigma_1, \tau_1} \cdots A_{\sigma_n, \tau_n} \\ &= \sum_{\sigma} \text{sign}(\sigma)^2 \det(A) \\ &= n! \det(A). \end{aligned}$$

这个定义可以通过 ... 推广到 Cayley 的“超行列式” (hyper determinants)。

一个有趣的性质是

$$\begin{array}{|c|c|c|} \hline A & \cdots & A \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline \cdots & & \\ \hline A & \cdots & A \\ \hline \end{array}$$

$$(18) \quad \det(A) = \prod_i \lambda_i \quad \dots$$

$$(19) \quad \det(cA) = c^n \det(A) \quad \begin{array}{|c|c|c|} \hline cA & \cdots & cA \\ \hline \end{array} = c^n \begin{array}{|c|c|c|} \hline A & \cdots & A \\ \hline \end{array}$$

$$(20) \quad \det(A) = \det(A^T) \quad \dots$$

$$(21) \quad \det(AB) = \det(A)\det(B) \quad \begin{array}{|c|c|c|} \hline A & \cdots & A \\ B & \cdots & B \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline A & \cdots & A \\ \hline B & \cdots & B \\ \hline \end{array}$$

$$(22) \quad \det(A^{-1}) = 1/\det(A) \quad \dots$$

$$(23) \quad \det(A^n) = \det(A)^n \quad \dots$$

$$(24) \quad \det(I + uv^T) = 1 + u^T v \quad \dots$$

6.2 逆矩阵

这里或许可以化简，除非余子式矩阵有漂亮的表示。

第七章 高级导数

7.1 向量范数的导数

7.1.1 二范数

$$\frac{d}{dx} \|x - a\|_2 = \frac{x - a}{\|x - a\|_2} \quad (7.1)$$

$$\frac{d}{dx} \frac{x - a}{\|x - a\|_2} = \frac{I}{\|x - a\|_2} - \frac{(x - a)(x - a)^T}{\|x - a\|_2^3} \quad (7.2)$$

$$\frac{d}{dx} \|x\|_2^2 = \frac{d}{dx} \|x^T x\|_2 = 2x \quad (7.3)$$

7.2 矩阵范数的导数

7.3 结构化矩阵的导数

7.3.1 对称矩阵

7.3.2 对角矩阵

7.3.3 Toeplitz 矩阵

7.4 行列式的导数

7.5 一般形式

7.6 线性形式

7.7 平形式

7.8 来自 Stack Exchange

令 $F(t) = |1 + tA|$, 则 $F'(t) = \text{Tr}(A)$ 且 $F''(t) = \text{Tr}(A)^2 - \text{Tr}(A^2)$.

7.9 逆矩阵的导数

7.9.1 迹恒等式

$$\frac{\partial}{\partial \mathbf{X}} \text{Tr}(\mathbf{A}\mathbf{X}^{-1}\mathbf{B}) = -(\mathbf{X}^{-1}\mathbf{B}\mathbf{A}\mathbf{X}^{-1})^T = -\mathbf{X}^{-T}\mathbf{A}^T\mathbf{B}^T\mathbf{X}^{-T}$$

假设 $\mathbf{B}$ 与 $\mathbf{C}$ 是对称矩阵, 则

$$\begin{aligned} \frac{\partial}{\partial \mathbf{X}} \text{Tr}[(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1}\mathbf{A}] &= -(\mathbf{C}\mathbf{X}(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1})(\mathbf{A} + \mathbf{A}^T)(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1} \\ \frac{\partial}{\partial \mathbf{X}} \text{Tr}[(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1}(\mathbf{X}^T\mathbf{B}\mathbf{X})] &= -2\mathbf{C}\mathbf{X}(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{B}\mathbf{X}(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1} \\ \frac{\partial}{\partial \mathbf{X}} \text{Tr}[(\mathbf{A} + \mathbf{X}^T\mathbf{C}\mathbf{X})^{-1}(\mathbf{X}^T\mathbf{B}\mathbf{X})] &= -2\mathbf{B}\mathbf{X}(\mathbf{X}^T\mathbf{C}\mathbf{X})^{-1} \\ &\quad - 2\mathbf{C}\mathbf{X}(\mathbf{A} + \mathbf{X}^T\mathbf{C}\mathbf{X})^{-1}\mathbf{X}^T\mathbf{B}\mathbf{X}(\mathbf{A} + \mathbf{X}^T\mathbf{C}\mathbf{X})^{-1} \\ &\quad + 2\mathbf{B}\mathbf{X}(\mathbf{A} + \mathbf{X}^T\mathbf{C}\mathbf{X})^{-1} \end{aligned}$$

7.10 特征值的导数

7.11 练习

练习 49. 求

$$f(x, \Sigma, \mu) = \frac{1}{\sqrt{(2\pi)^k |\Sigma|}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

关于 Σ 的导数。

第八章 特殊张量

除了秩或阶之外，有些张量还具有特殊结构，从而拥有一些很好的性质，例如可以快速缩并。下面给出若干例子。

8.1 快速 Kronecker 乘法

假设我们想计算 $(A_1 \otimes A_2 \cdots A_n)x$ ，其中 A_i 是 $a_i \times a_i$ 矩阵，且 $x \in \mathbb{R}^{a_1 a_2 \cdots a_n}$ 。如果先计算 Kronecker 积，再做矩阵-向量乘法，就需要 $(a_1 \cdots a_n)^2$ 的时间。

相反，我们可以把 x 重塑为一个 $a_1 \times \cdots \times a_n$ 张量，并执行乘法

$$\begin{array}{c} \begin{array}{c} \frac{a_1}{-} A_1 \\ \frac{a_2}{-} A_2 \\ \vdots \\ \frac{a_n}{-} A_n \end{array} \begin{array}{c} \diagup \\ \diagdown \\ \diagup \\ \diagdown \end{array} \begin{array}{c} a_1 \\ \\ \\ a_n \end{array} X \end{array}$$

也就是逐一缩并这些 a_i 边。这需要时间

$$a_1^2(a_2 \cdots a_n) + a_2^2(a_1 a_3 \cdots a_n) + \cdots + a_n^2(a_1 \cdots a_{n-1}) = (a_1 + \cdots + a_n)(a_1 \cdots a_n),$$

正如后面会看到的，这是许多快速算法的基础。

8.2 Hadamard 变换

Hadamard 矩阵定义为 $H_{2^n} = H_2^{\otimes n} = H_2 \otimes \cdots \otimes H_2$ 其中 $H_2 = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ 。例如

$$H_4 = H_2 \otimes H_2 = \begin{bmatrix} H_2 & H_2 \\ H_2 & -H_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.$$

$$H_{2^n} = \begin{array}{c} \text{---} \triangleleft \begin{array}{c} H_2 \\ \vdots \\ H_7 \end{array} \triangleright \text{---} \end{array}.$$
$$H_{2^n}x = \begin{bmatrix} H_{2^{n-1}} & H_{2^{n-1}} \\ H_{2^{n-1}} & -H_{2^{n-1}} \end{bmatrix} \begin{bmatrix} x^{(1)} \\ x^{(2)} \end{bmatrix},$$

另一种做法是直接使用上面描述的一般事实，其中所有 $a_i = 2$ 。于是“快速 Kronecker 乘法”方法需要时间 $(a_1 a_2 \cdots a_n)(a_1 + a_2 + \cdots a_n) = 2n \log_2 n$ 。

离散 Fourier 矩阵由 $(F_N)_{i,j} = \omega^{ij}$ 定义, 其中 $\omega = e^{-2\pi i/N}$:

$$F_N = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \dots & \omega^{N-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{2(N-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{3(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{N-1} & \omega^{2(N-1)} & \omega^{3(N-1)} & \dots & \omega^{(N-1)(N-1)} \end{bmatrix}.$$

$$\text{exp} \leftarrow \left[\begin{array}{c} n \\ n \end{array} \right]$$

The diagram illustrates a two-stage network F_N . It consists of two rectangular blocks, P_1 and P_2 , connected in series. Block P_1 has n_1 inputs and n_2 outputs. Block P_2 has n_2 inputs and n_3 outputs. The inputs to P_1 are labeled $F_{p_1}, F_{p_2}, \dots, F_{p_{n_1}}$. The outputs of P_1 are labeled $F_{p_1}, F_{p_2}, \dots, F_{p_{n_2}}$. The inputs to P_2 are labeled $F_{p_1}, F_{p_2}, \dots, F_{p_{n_2}}$. The outputs of P_2 are labeled $F_{p_1}, F_{p_2}, \dots, F_{p_{n_3}}$.

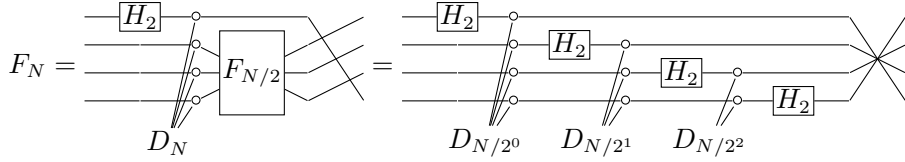
其中 $N = p_1^{i_1} p_2^{i_2} \cdots p_n^{i_n}$ 是 N 的素因数分解, P_1 和 P_2 是某些置换矩阵。

使用快速 Kronecker 乘法, 该算法需要 $(p_1^{i_1} + \cdots + p_n^{i_n})N$ 时间。通过给 x 补零, 可以把 N 增大一个常数倍, 从而得到一串 $n = O(\log(N)/\log \log(N))$ 个素数, 其和为 $\sim n^2/\log n = O(\log(N)^2)$ 。因此完整算法需要 $O(N \log(N)^2)$ 时间。接下来我们会看到如何把它降低到 $O(N \log N)$ 。

经典的 Cooley-Tukey FFT 算法使用如下递归:

$$F_N = \begin{bmatrix} I & I \\ I & -I \end{bmatrix} \begin{bmatrix} I & 0 \\ 0 & D_{N/2} \end{bmatrix} \begin{bmatrix} F_{N/2} & 0 \\ 0 & F_{N/2} \end{bmatrix} \begin{bmatrix} \text{even-odd} \\ \text{permutation} \end{bmatrix},$$

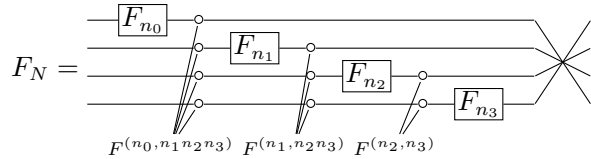
其中 $D_N = [1, w^N, w^{2N}, \dots]$ 。偶奇置换会把所有偶数位置的值移到开头。如果把 I_{2^n} 重塑为 $I_2 \otimes \cdots \otimes I_2$, 这个置换就是 $P_N = \text{---}$, 或者用 pytorch 写作: `x.permute([3, 0, 1, 2])`。还要注意, $\begin{bmatrix} I & I \\ I & -I \end{bmatrix} = H_2 \otimes I$ 且 $\begin{bmatrix} F_{N/2} & 0 \\ 0 & F_{N/2} \end{bmatrix} = I_2 \otimes F_{N/2}$ 。因此可以用张量图记号写成:



由于可以在线性时间内完成置换矩阵和对角矩阵的乘法, 所以 $O(n \log n)$ 的时间复杂度可以由 Hadamard 情形中的同一论证推出。

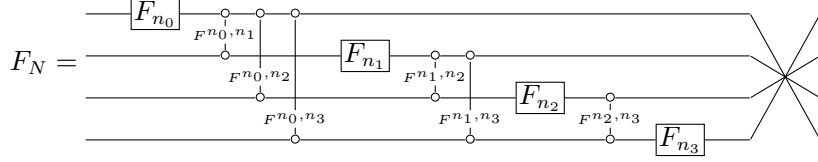
注意这里有许多对称性; 例如, 由于该矩阵是对称的, 可以通过转置 (水平翻转) 得到对称形式。也可以把置换推到左侧。

我们不必把矩阵二等分, 也可以把它三等分、四等分, 等等。使用这种广义 Cooley-Tukey 算法, 可以得到下图:



其中 $n_0 n_1 n_2 n_3 = N$ 。这里我们把 D_N 矩阵替换成了“广义”Fourier 矩阵 $F^{(a,b)}$; 它定义为 $F_{j,k}^{(a,b)} = e^{-2\pi i j k / (ab)}$, 并按需要进行了重塑。在只分成两个大致满足 $n_1 = n_2 = \sqrt{N}$ 的部分这一简单情形中, 它也称为“四步 FFT” (Four step FFT) 或 Bailey 的 FFT 算法。

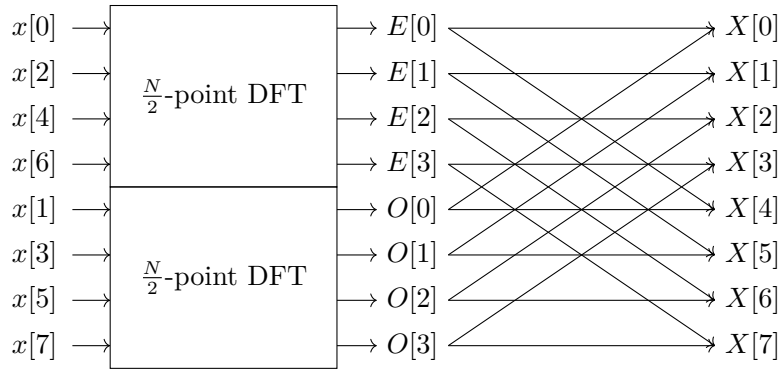
可以利用性质 $F_{a,bc} = F_{a,b} \bullet F_{a,c}^{1/b}$ 进一步化简该图：



在每次都分成 2 份的简单情形中，这也称为“量子 FFT”（Quantum FFT）算法。

上面隐藏了一些细节，也就是说这些矩阵应当除以不同的 N 。

注意，这个图可能与你见过的一些 FFT 图不同。那些图通常长这样：



并且有 2^n 行。而张量图只有 n 行（也就是 $\log_2 N$ 行）。

多维 Fourier 变换

这只是沿每个轴分别做 Fourier 变换。

8.3.1 Vandermonde 矩阵

可以定义“Vandermonde 函数”（Vandermonde Function） $\text{vm}_N(x) = (1, x, x^2, \dots, x^{N-1})$ 。于是 Vandermonde 矩阵 $V_{i,j} = x_i^j$ 定义为：

$$- \text{vm} \langle \cdots x -$$

如果 $a \in \mathbb{R}^N$ 是多项式 $P(x) = a_0 + a_1x + \dots$ 的系数向量，那么可以如下得到“批量求值”（batch evaluation）向量 $(P(x_1), P(x_2), \dots)$ ：

$$a - \text{vm} \langle \cdots x -$$

使用上面描述的 FFT，可以快速完成这一过程（时间为 $O(n(\log n)^2)$ ）。思路是定义两个多项式，它们分别在前半部分点和后半部分点上为零：

$$m_0(x) = (x - x_1) \cdots (x - x_{n/2}) \quad \text{且}$$

$$m_1(x) = (x - x_{n/2+1}) \cdots (x - x_n).$$

然后使用多项式余数定理计算 $R_0 = P \bmod m_0$ 和 $R_1 = P \bmod m_1$ ；借助 FFT，这可以在 $O(n \log n)$ 时间内完成。这意味着

$$P(x) = Q_0(x)m_0(x) + R_0(x) \quad \text{且}$$

$$P(x) = Q_1(x)m_1(x) + R_1(x)$$

按构造成立，其中 R_0 和 R_1 是次数至多为 $n/2$ 的多项式。由于 m_0 和 m_1 的定义方式，有

$$R_0(x_i) = P(x_i) \quad \text{对 } i \leq n/2 \quad \text{且}$$

$$R_1(x_i) = P(x_i) \quad \text{对 } i > n/2.$$

因此，要在所有 n 个 x_i 上计算 P ，只需分别在两半点上计算较小的多项式 R_0 和 R_1 。（多项式 Q_0 和 Q_1 可以忽略。）这给出了一个满足 $T(n) = 2T(n/2) + n \log n$ 的分治算法，从而推出 $T(n) = O(n(\log n)^2)$ 。

8.3.2 块矩阵

Schur 补这类对象很有意思。但我们能否用张量图说出一些有用的内容？

来自 The Matrix Cookbook：考虑矩阵

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

上面矩阵中块 A_{11} 的 Schur 补是如下矩阵（在上面的文字中记为 C_2 ）

$$A_{22} - A_{21}A_{11}^{-1}A_{12}$$

上面矩阵中块 A_{22} 的 Schur 补是如下矩阵（在上面的文字中记为 C_1 ）

$$A_{11} - A_{12}A_{22}^{-1}A_{21}$$

利用 Schur 补，可以把块矩阵的逆改写为

$$\left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right]^{-1} = \left[\begin{array}{c|c} I & 0 \\ \hline -A_{22}^{-1}A_{21} & I \end{array} \right] \left[\begin{array}{c|c} (A_{11} - A_{12}A_{22}^{-1}A_{21})^{-1} & 0 \\ \hline 0 & A_{22}^{-1} \end{array} \right] \left[\begin{array}{c|c} I & -A_{12}A_{22}^{-1} \\ \hline 0 & I \end{array} \right]$$

在求解如下形式的线性方程组时，Schur 补很有用：

$$\left[\begin{array}{c|c} A_{11} & A_{12} \\ \hline A_{21} & A_{22} \end{array} \right] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}$$

它给出关于 x_1 的如下方程：

$$(A_{11} - A_{12}A_{22}^{-1}A_{21})x_1 = b_1 - A_{12}A_{22}^{-1}b_2$$

当相应的逆存在时，可以先解出 x_1 ，再把它代入关于 x_2 的方程以解出 x_2 。

8.3.3 对称、斜对称/反对称

可以在这里介绍 Penrose 的对称张量？

8.3.4 Toeplitz 矩阵

可以在这里讨论卷积张量……

8.3.5 单位矩阵、置换与移位

不是特别有意思……

第九章 分解

9.1 高阶奇异值分解

假设有一个 n 阶张量 A 。我们沿每个维度“展开” (unfold) A 。这意味着把边 i 拉到左侧，并把其余部分展平到右侧。然后计算 SVD，即 USV 。这里 U 是一个方阵，我们会保留它。用 U^T 乘到 A 的第 i 条边上（它同时也是 U 的逆）。结果会得到一个“核心” (core) 张量以及一系列 U 张量。如果想要更紧凑的 SVD，可以像普通 SVD 一样令每个 U 低秩。还有一种“交错计算” (Interlacing computation)，即在遍历过程中不断把各个 U^T 乘到 A 上。

对于 3 阶张量，这种方法称为“Tucker 分解” (Tucker decomposition)。

如果“核心矩阵”是对角的，这就称为张量秩分解。如果我们能很好地完成这种分解，就可以用它分解 $I^{\otimes 3}$ ，从而得到更好的矩阵乘法算法。遗憾的是，张量秩分解是 NP-hard 的。

我猜如果对核心张量做对角化，HOSVD 会给出一个秩分解。只是它不会是一个高效分解。

9.2 秩分解

9.2.1 边界秩

张量的边界秩是与它接近的张量中最小的秩。TODO: 给出边界秩远小于秩的例子。

9.3 快速矩阵乘法

Strassen 定义了 3 个形状为 $7 \times 2 \times 2$ 的张量:

$$S_A = \begin{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, & \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 1 \\ 0 & -1 \end{bmatrix} \end{bmatrix}$$

$$S_B = \begin{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 1 \\ 0 & -1 \end{bmatrix}, & \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix} \end{bmatrix}$$

$$W = \begin{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 0 & 1 \\ 0 & -1 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, & \begin{bmatrix} -1 & 0 \\ 1 & 0 \end{bmatrix}, & \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, & \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \end{bmatrix}$$

这些张量具有一个很好的性质: 它们分解了 $I_2 \otimes I_2 \otimes I_2$:

$$\text{Diagram: } S_A \otimes S_B \otimes W = I_2 \otimes I_2 \otimes I_2$$

为了以快于通常 n^3 时间的方式相乘两个矩阵 A 和 B , 我们把它重塑为形状为 $(2, \frac{n}{2}, 2, \frac{n}{2})$ 的块矩阵, 并使用 Strassen 的张量:

$$= A = B = \text{Diagram: } A \otimes B \otimes \text{Strassen Tensor}$$

按正确顺序缩并这些边, 只需要 $7/8n^3 + O(n^2)$ 次运算。

如果改为重塑成 $(2, 2, \dots, 2)$,

$$AB = \text{Diagram: } A \otimes B$$

并沿每个轴使用 Strassen 张量, 那么工作量会减少 $(7/8)^{\log_2(n)}$, 从而得到时间为 $n^{3+\log_2(7/8)} = n^{2.80735}$ 的矩阵乘法。

缩并双边 $S_A - A$ 和 $S_B - B$ 都需要 $O(n^2)$ 时间。

还需要验证这确实比朴素矩阵乘法更快：缩并 $S_A - A$ 需要 $7 \cdot 2^2(n/2)^2$ 次运算， $S_B - B$ 同理。接下来缩并 $S_A A - S_B B$ ，这需要 $7(n/2)^3$ 时间。最后缩并与 W 相连的边，需要 $2^2 \cdot 7(n/2)^2$ 。关键项是三次项 $7/8n^3$ ；如果改为递归执行，就得到经典的 $O(n^{\log_2 7})$ 算法。

FIXME：什么是“与 W 相连的边”？我认为我们必须/想要立即缩并与 W 相连的超边？

其他

如果改用形状为 (n, m) 和 (m, p) 的块来表示 A 和 B ，就可以分解 $I_n \otimes I_m \otimes I_p$ ，并用与上面 Strassen $(2, 2, 2)$ 张量相同的方法得到矩阵乘法算法。许多论文研究过这个问题，并由此产生了 Josh Alman 等人的最佳算法。例如，Deep Mind 找到了 $I_3 \otimes I_4 \otimes I_5$ 的秩 47 分解。

也许更有趣的例子是 $(4, 4, 4)$ 张量，他们为它找到了一个秩 47 分解。一种容易构造秩 49 分解的方法是取 Strassen 分解并把它加倍。这会是一个值得展示的例子吗？也许太杂乱了？不过实际上，他们的秩 47 构造只在“模”（modular）情形下成立。于是 $(3, 4, 5)$ 是一般情形。

第十章 机器学习应用

10.1 最小二乘

10.2 交叉熵损失的 Hessian 矩阵

10.3 卷积神经网络

10.4 Transformer / 注意力

10.5 张量草图

10.6 强化学习

在考虑不完全信息博弈时，我们尝试用一个低秩张量来近似每个玩家的策略。可以用张量草图来近似这个张量。基本上，这意味着选取一个随机向量外积，并将其乘到导数上。这应该会很快……

第十一章 张量算法

11.1 张量缩并顺序

在前面的章节中，我们画了大量张量网络。虽然理论层面对某些应用已经足够，但确实存在一些研究领域，主要是量子线路模拟这类计算型领域，需要真正计算张量网络背后的张量。这通过所谓的张量缩并来完成。为了理解它的含义，考虑下面这个 2-张量图，其中我们缩并两个张量 A 和 B ：

$$\begin{array}{c} \text{---} \underset{2}{A} \text{---} \underset{4}{B} \text{---} \end{array} \xrightarrow{\text{Contraction}} \begin{array}{c} \text{---} \underset{2}{A'} \text{---} \end{array}$$

注意，我们还标出了各个维度的大小，也就是说， A 是一个 $2 \times 2 \times 4$ 张量，而 B 是一个 4×2 矩阵。现在，缩并成本定义为缩并过程中执行的 FLOPS 数；按照惯例，这里指的是标量乘法次数。¹ 在这个例子中，缩并成本为 $2 \times 2 \times 4 \times 2 = 32$ ，也就是说，只需把维度大小相乘。

前面的例子相当小。然而，实际中的张量网络往往包含数百个张量。自然，这些张量也需要缩并成单个张量。有趣的地方来了：执行这些缩并的顺序可能会对运行时间产生巨大影响。为了对此建立直觉，考虑一个扩展的 3-张量图例子：

$$\begin{array}{c} \text{---} \underset{2}{A} \text{---} \underset{4}{B} \text{---} \underset{1}{C} \text{---} \end{array}$$

如果先缩并 A 和 B (再把所得张量与 C 缩并), 成本为 $2^2 \times 4 \times 2 + 2^3 \times 1 \times 2^2 = 32 + 32 = 64$ 次乘法；而如果一开始就缩并 B 和 C ，总共会得到 $2^3 \times 1 \times 2^2 + 2^2 \times 4 \times 2^3 = 32 + 128 = 160$ 次乘法。因此，第一种顺序要好得多。

于是很自然地会问：我们总能找到最优缩并顺序吗？

¹ 这假设该操作采用朴素实现，也就是在各个维度上使用嵌套循环。

11.1.1 算法

下面总结一些用于寻找良好缩并顺序的知名结果和框架。

最优算法

事实上，为任意形状的张量网络寻找最优缩并顺序相当困难；更准确地说，它是 NP-hard 的 [3]。有一个著名的精确算法运行时间为 $O(3^n)$ [17]，其中 n 是网络中的张量数量。它能找到相对于总缩并成本而言的最优缩并顺序。如果关心的是最小化最大中间张量的大小，也就是优化执行过程中的内存使用，那么可以在 $O(2^n n^3)$ 时间内更快地完成 [22]。

好消息是，对于某些受限形状的张量网络，确实存在高效算法。经典例子是矩阵链问题的动态规划解法 [4]，它其实就是我们的问题，只不过只针对矩阵。朴素算法运行时间为 $O(n^3)$ ，但可以实现为 $O(n^2)$ 时间 [26]（甚至可以做到 $O(n \log n)$ 时间 [11, 12]）。另一类存在多项式时间算法的形状是树张量网络 [25, 23]。

另一种重要的缩并顺序优化方法，是对张量网络的线图表示做树分解 [15, 5, 19]。特别地，这会得到一个缩并顺序，其最大中间张量秩等于该树分解的树宽。粗略地说，树宽衡量一个图有多像树。这并不能直接解决我们的问题，因为寻找最小树宽的树分解本身也是困难的 [14]。两个用于寻找良好树分解的知名框架是 QuickBB [6] 和 FlowCutter [24, 9]。

尽力而为算法

然而，一旦我们想缩并任意网络形状，能做的最好事情就是退回到启发式或近似方法。两个知名框架是 `opt_einsum` [20] 和 `cotengra` [7]，它们旨在优化任意 `einsum` 的缩并顺序（也称为“缩并路径”（contraction path））：对于最优算法会过慢的张量网络，`opt_einsum` 会应用一种特设的贪心算法，而 `cotengra` 使用超图划分 [18]，并结合一种后来得到改进的 Bayesian 优化方法 [21]。其他借鉴数据库查询优化的算法则在 `netzwerk` [23] 中实现。另一种选择是学习最佳缩并顺序，例如使用强化学习 [16]。

自然，上述启发式方法并不带有最优性保证。存在一个运行时间为 $O^*(2^n/\epsilon)$ 的 $(1 + \epsilon)$ -近似算法，可以最小化中间张量大小之和 [22]。

值得一提的是 `einsum` 的 SQL 视角 [2]：它允许在任意数据库引擎上执行整个张量网络缩并。事实上，对于某些 `einsum` 类，在 Hyper [13] 等先进数据库引擎上运行缩并，可能比使用事实标准的 `numpy` 数组实现快得多。

第十二章 Tensorgrad

实现细节

12.1 同构

实际上存在“张量同构”（tensor isomorphism）的概念，但它基本上与图同构相同。

我们需要在代码的许多不同部分理解同构。

12.1.1 在乘积中

- 消去 / 合并乘积中的相同部分这其实格外困难，因为必须收集构成同构子图的一组节点。目前我们稍微取巧，只考虑乘积中的独立连通分量。

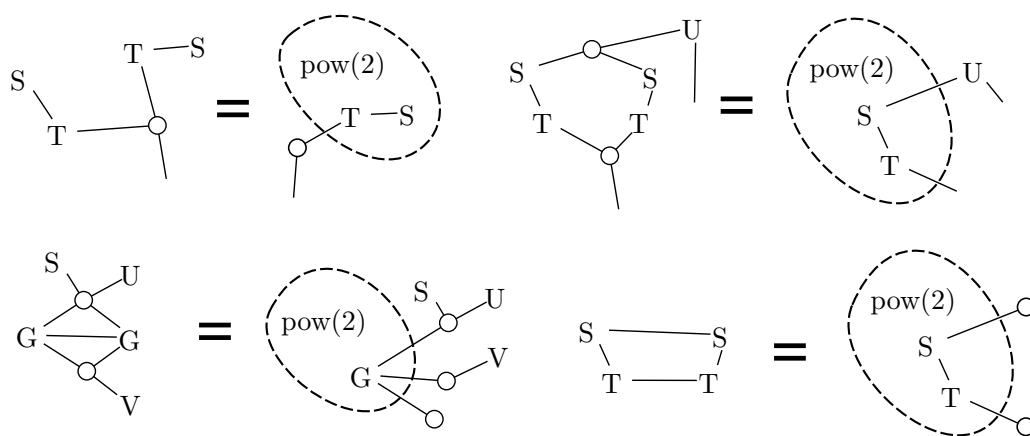


图 12.1: 合并乘积中的相同部分

这个问题基本上是：

1. 给定一个多重图 G ，其节点为 V 、边为 E 。
2. 节点和边都有标签。
3. 要找到 V 的两个不相交子集 V_1 和 V_2 ，使得由 V_1 和 V_2 诱导出的子图 G_1 和 G_2 同构。此外，在该同构下， G_1 和 G_2 中节点与边的标签相同。

这个问题很可能是 NP-hard 的，但也许仍然存在比尝试所有子集的 2^n 更快的算法。特别是，我们可以修改 VF2 算法；该算法会迭代地尝试匹配 G_1 和 G_2 中的节点。NetworkX 库已经有一个 GraphMatcher，可以搜索同构子图。它也许可以扩展到我们的问题……但老实说，我不知道我们是否真的想在最一般的情形下解决这个问题，因为它有点对应于对图做因式分解。而我们并不做因式分解，就像我们不做逆分配律一样。

无论如何，显然我们需要能够按同构来比较节点和边。

此外，乘积中同构规范化的基本用例，只是根据各个部分计算乘积本身的规范形式。这里我们方法的一部分，是把外部边转成节点，从而可以给它们着色。

12.1.2 在求和中

在判断 $A+B$ 是否等于 $2A$ 时，需要检查 A 和 B 是否同构。但我们还必须在当前边重命名下完成这件事。这就是为什么不能直接把 $A+A^T$ 变成 $2A$ 。

我的代码中实际的工作方式是

```

1 def key_fn(t: Tensor):
2     # Align tensor edges to have the same order, using Sum's order as reference.
3     canons = [t.canonical_edge_names[t.edges.index(e)] for e in self.edges]
4     return hash((t.canon,) + tuple(canons))
5
6 ws_tensors = TensorDict(key_fn=key_fn, default_fn=int)
7 for w, t in zip(weights, tensors):
8     ws_tensors[t] += w
9 ws_tensors = [(w, t) for t, w in ws_tensors.items()]

```

这表示我用于 hash 的内容，是张量的规范形式，加上按照求和中边顺序排列的边的规范形式。这些基本上就是轨道；这意味着如果被求和的张量具有某种对称性，我们就允许“翻转”它，从而使各项同构。

在“compute canonical”方法中，我们做的事情大致相同，但还会包含权重。

```

1 def _compute_canonical(self):
2     hashes = []
3     for e in self.edges:
4         canons = [t.canonical_edge_names[t.edges.index(e)] for t in self.tensors]

```

```

5     hashes.append(hash(("Sum",) + tuple(sorted(zip(self.weights, canons)))))
6     base = hash(("Sum", len(self.tensors)))
7     hashes = [hash((base, h)) for h in hashes]
8     return base, hashes

```

将来我们想改用对称群。求和的对称群会是什么？它是各个求和项对称群乘积的对角部分。怎样才能找到这个群的生成元？也许应该直接构造某个联合图，然后寻找该图的自同构。

另一种选择是使用 sympy。目前尚不清楚这个问题是否可以在多项式时间内求解。我认为 Babai 证明了它是准多项式的，但该算法并不实用。顺带一提，子群交、元素中心化子以及 $\{1, \dots, n\}$ 子集稳定子这些问题已经被证明（由 Eugene Luks 证明）在多项式意义下等价。

实际上，构造一个图并使用 nauty 是个很好的想法，因为它能够检测出 $A + A^T$ 是对称的。仅取各个求和项自同构群的交集并不能发现这一点。

另一种选择是把求和转换成一个函数……但不，这有点奇怪。那会要求我支持任意数量输入的函数，而目前还不是这样。

12.1.3 在求值中

在对张量求值时，可以查看该张量的图，并判断它是否同构于先前已经求值过的某个张量。这是一个并不真正需要规范形式的例子；近似 hash 加上 vf2 就足够了。还要注意，在这种情况下我们并不关心边的重命名，因为可以在返回张量之前直接重命名边。例如，如果已经求值过 A ，就可以很容易用它得到 A^T 。

12.1.4 在变量中

在变量中，我们把变量名包含进 hash。基本上，我们假设同名变量指向同一份数据。

```

1     base = hash(("Variable", self.name))
2     return base, [hash((base, e)) for e in self.original_edges]

```

对于原始规范边名，我们使用重命名前的边名。这意味着在 A^T 的情形下，它会与 A 具有相同的 hash。但因为它已经被重命名，Sum 中的 `t.index` 调用会翻转这些边。

可以设想让变量接收一个自同构群作为参数，这样就能定义具有不同对称性的变量。例如一个对称矩阵 A ，此时 $A + A^T$ 实际上就是 $2A$ 。

12.1.5 在常量中

在计算 Zero 或 Copy 这类常量的规范形式时，我们并不关心边名。我猜这是因为我们使用的常量都是最大对称的？目前我们会包含常量的 `tag`；如果它来自某个变量，那么这个 `tag` 就是该变量的 `hash`。

12.1.6 在函数中

一个问题是，原始名称通常是函数定义的一部分，而求导新增的边往往是根据上下文自动生成的，因此它们其实不应该成为规范形式的一部分。

与 Sum 不同，这里我们不会对规范名排序，因为输入顺序是重要的。

也许应该允许函数变换对称群？例如，如果有一个函数接收对称矩阵并返回对称矩阵，那么我们应当能使用输入的对称群来化简输出。

12.1.7 在导数中

我们所做的只是对张量和 `wrt` 做 `hash`。然后为导数添加新边。

12.1.8 其他

对于某些张量，可能存在并非等价关系的边维度关系。例如，一个 `flatten` 张量会让 “out” 边维度等于各个 “in” 边维度的乘积。

在之前的版本中，我让每个张量都注册一个 “回调” (`callback`) 函数。每当某个边维度 “变得可用” 时，该张量就有机会发出新的边维度。然而，这会给每个张量的实现增加很多工作，而且现有张量都不需要它。

12.2 重命名

这是代码中一个重要部分。

12.3 求值

求值中的一个重要部分是确定每条边的维度。为此，我基本上会构造张量的完整图，并使用一个名为 `edge_equivalences` 的函数，它给出一列元组 $((t_1, e_1), (t_2, e_2))$ ，表示张量 t_1 的边 e_1 等价于张量 t_2 的边 e_2 。注意，同一个边名可能在图中出现多次，因此也需要跟踪张量本身。

对于变量，由于用户以变量形式给出边维度，因此跟踪重命名后的边名很重要：

```

1 for e1, e2 in zip(self.original_edges, self.edges):
2     yield (self, e1), (self, e2)

```

对于常量，可能存在一些基于该常量来源张量的等价关系。

```

1 def edge_equivalences(self):
2     if self.link is not None:
3         yield from self.link.edge_equivalences()
4         for e in self.link.edges:
5             if e in self.edges:
6                 yield (self, e), (self.link, e)

```

对于复制张量，所有边都是等价的：

```

1 def edge_equivalences(self):
2     yield from super().edge_equivalences()
3     for e in self.edges[1:]:
4         yield (self, self.edges[0]), (self, e)

```

对于函数，我们实际上无法对函数自身的边（`self.edges_out`）说什么，但至少可以对广播边说一些事情。

```

1 for t, *inner_edges in self.inputs:
2     yield from t.edge_equivalences()
3     for e in t.edges:
4         if e not in inner_edges:
5             yield (t, e), (self, e)

```

也许还可以说同名输入边是等价的？

对于乘积，我们查看每条边 (t_1, e, t_2) ，并产生 $(t_1, e), (t_2, e)$ 。不过对于自由边 (t, e) ，我们会把它们与自身匹配，即 $(t, e), (self, e)$ 。

```

1 def edge_equivalences(self):
2     pairs = defaultdict(list)
3     for t in self.tensors:
4         yield from t.edge_equivalences()
5         for e in t.edges:
6             pairs[e].append(t)
7     for e, ts in pairs.items():
8         if len(ts) == 1:
9             yield (self, e), (ts[0], e)
10        else:
11            t1, t2 = ts
12            yield (t1, e), (t2, e)

```

类似地，对于求和，一切都只是与自身匹配：

```
1 def edge_equivalences(self):
2     for t in self.tensors:
3         yield from t.edge_equivalences()
4         for e in t.edges:
5             yield (t, e), (self, e)
```

最后，我们使用 BFS 将边维度从变量（由用户给定）传播到图的其余部分。

为什么非变量也需要知道边维度？主要是因为复制张量：我们会用它们表示超边，并且必须构造它们。如果我们不用 copy 而是更高效地计算超边，能否摆脱这一点？有时也会出现“分离的”（detached）copy……

此外，另一种想法是真正构造完整图。我原本认为这不可行，因为 Sum 并不真的是图。但也许借助使用 nauty 的新方法，我们实际上可以做到这一点。

12.3.1 乘积

我们只需对乘积中的张量求值，并把它们交给 einsum。

12.4 化简规则

这类规则有很多。

大多数情况下，我们可以在一次深度优先遍历中完成所有事情，但有少数时候需要多轮遍历。这可以通过 full-simplify 方法完成；它会反复调用 simplify，直到不再发生变化。

参考文献

- [1] John Aldrich. *The Origins of Mathematical Words: A Comprehensive Dictionary of Latin, Greek, and Arabic Roots*. Mathematical Association of America, 2010.
- [2] Mark Blacher, Julien Klaus, Christoph Staudt, Sören Laue, Viktor Leis, and Joachim Giesen. Efficient and portable einstein summation in SQL. *Proc. ACM Manag. Data*, 1(2):121:1–121:19, 2023.
- [3] Lam Chi-Chung, P Sadayappan, and Rephael Wenger. On optimizing a class of multi-dimensional loops with reduction for parallel execution. *Parallel Processing Letters*, 7(02):157–168, 1997.
- [4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, Third Edition*. The MIT Press, 3rd edition, 2009.
- [5] Jeffrey M Dudek, Leonardo Duenas-Osorio, and Moshe Y Vardi. Efficient contraction of large tensor networks for weighted model counting through graph decompositions. *arXiv preprint arXiv:1908.04381*, 2019.
- [6] Vibhav Gogate and Rina Dechter. A complete anytime algorithm for treewidth. In David Maxwell Chickering and Joseph Y. Halpern, editors, *UAI '04, Proceedings of the 20th Conference in Uncertainty in Artificial Intelligence, Banff, Canada, July 7-11, 2004*, pages 201–208. AUAI Press, 2004.
- [7] Johnnie Gray and Stefanos Kourtis. Hyper-optimized tensor network contraction. *Quantum*, 5:410, 2021.
- [8] greg (<https://math.stackexchange.com/users/357854/greg>). Proving that $\text{vec}(a \text{diag}(b) c) = ((c^t \otimes 1_a) \odot (1_c \otimes a)) b$. Mathematics Stack Exchange. URL:<https://math.stackexchange.com/q/2993406> (version: 2018-11-11).

- [9] Michael Hamann and Ben Strasser. Graph bisection with pareto optimization. *Journal of Experimental Algorithmics (JEA)*, 23:1–34, 2018.
- [10] William Rowan Hamilton. *Lectures on Quaternions*. Hodges and Smith, 1853.
- [11] T. C. Hu and M. T. Shing. Computation of matrix chain products. part I. *SIAM J. Comput.*, 11(2):362–373, 1982.
- [12] T. C. Hu and M. T. Shing. Computation of matrix chain products. part II. *SIAM J. Comput.*, 13(2):228–251, 1984.
- [13] Alfons Kemper and Thomas Neumann. Hyper: A hybrid oltp&olap main memory database system based on virtual memory snapshots. In *2011 IEEE 27th International Conference on Data Engineering*, pages 195–206. IEEE, 2011.
- [14] Tuukka Korhonen. A single-exponential time 2-approximation algorithm for treewidth. *SIAM Journal on Computing*, (0):FOCS21–174, 2023.
- [15] Igor L Markov and Yaoyun Shi. Simulating quantum computation by contracting tensor networks. *SIAM Journal on Computing*, 38(3):963–981, 2008.
- [16] Eli Meirom, Haggai Maron, Shie Mannor, and Gal Chechik. Optimizing tensor network contraction using reinforcement learning. In *International Conference on Machine Learning*, pages 15278–15292. PMLR, 2022.
- [17] Robert N. C. Pfeifer, Jutho Haegeman, and Frank Verstraete. Faster identification of optimal contraction sequences for tensor networks. *Phys. Rev. E*, 90:033315, 2014.
- [18] Sebastian Schlag, Vitali Henne, Tobias Heuer, Henning Meyerhenke, Peter Sanders, and Christian Schulz. K-way hypergraph partitioning via n-level recursive bisection. In *2016 Proceedings of the Eighteenth Workshop on Algorithm Engineering and Experiments (ALENEX)*, pages 53–67. SIAM, 2016.
- [19] Roman Schutski, Danil Lykov, and Ivan Oseledets. Adaptive algorithm for quantum circuit simulation. *Phys. Rev. A*, 101:042335, Apr 2020.
- [20] Daniel G. A. Smith and Johnnie Gray. opt_einsum - a Python package for optimizing contraction order for einsum-like expressions. *Journal of Open Source Software*, 3(26):753, 2018.

- [21] Christoph Staudt, Mark Blacher, Julien Klaus, Farin Lippmann, and Joachim Giesen. Improved cut strategy for tensor network contraction orders. In Leo Liberti, editor, *22nd International Symposium on Experimental Algorithms, SEA 2024, July 23-26, 2024, Vienna, Austria*, volume 301 of *LIPICs*, pages 27:1–27:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [22] Mihail Stoian and Andreas Kipf. Dpconv: Super-polynomially faster join ordering, 2024.
- [23] Mihail Stoian, Richard M. Milbradt, and Christian B. Mendl. On the optimal linear contraction order of tree tensor networks, and beyond. *SIAM Journal on Scientific Computing*, 46(5):B647–B668, 2024.
- [24] Ben Strasser. Computing tree decompositions with flowcutter: PACE 2017 submission. *CoRR*, abs/1709.08949, 2017.
- [25] Jianyu Xu, Ling Liang, Lei Deng, Changyun Wen, Yuan Xie, and Guoqi Li. Towards a polynomial algorithm for optimal contraction sequence of tensor networks from trees. *Phys. Rev. E*, 100:043309, 2019.
- [26] F. Frances Yao. Efficient dynamic programming using quadrangle inequalities. In Raymond E. Miller, Seymour Ginsburg, Walter A. Burkhard, and Richard J. Lipton, editors, *Proceedings of the 12th Annual ACM Symposium on Theory of Computing, April 28-30, 1980, Los Angeles, California, USA*, pages 429–435. ACM, 1980.

第十三章 附录

这里包含一些证明，例如公式 524 或 571 的证明。这些证明相当长，但可以用来和图示证明作对比。